

NATIONAL ENGINEERING CENTER
University of the Philippines

2.0

Deep Learning, Transformers, and LLM Foundations

Eugene Rex L. Jalao PhD

Professor

Department of Industrial Engineering and Operations Research

University of the Philippines Diliman

eljalao@up.edu.ph

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Module Outline

- Introduction to Agentic AI
  - Capstone Project Case Proposal Generation
- **Deep Learning, Transformers and LLMs**
  - **Capstone Project Business Case Workflow**
- Agentic Workflows
  - Case Study and Capstone Project Work
- Agentic RAGs
  - Case Study and Capstone Project Work
- Orchestration
  - Case Study and Capstone Project Work
- Stateful Multi-Agent Systems
  - Case Study and Capstone Project Work
- Evaluation
  - Case Study and Capstone Project Work
- LLMOps
  - Capstone Project Presentation

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Outline for this Session

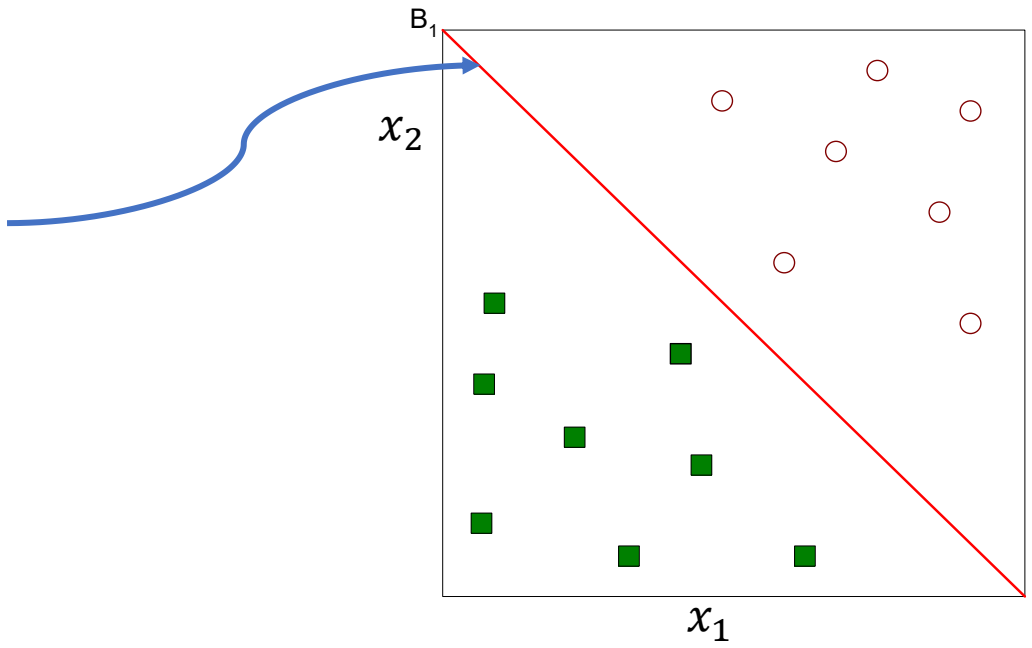
- Introduction to ANNs
- Transformers
- Embeddings
- Attention
- Quantization
- Project Case

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Perceptron

- Let a binary $y = Green\ or\ Red$
- Given data
  - Find a line that separates the data

$$b + w_1 x_1 + w_2 x_2 = z$$

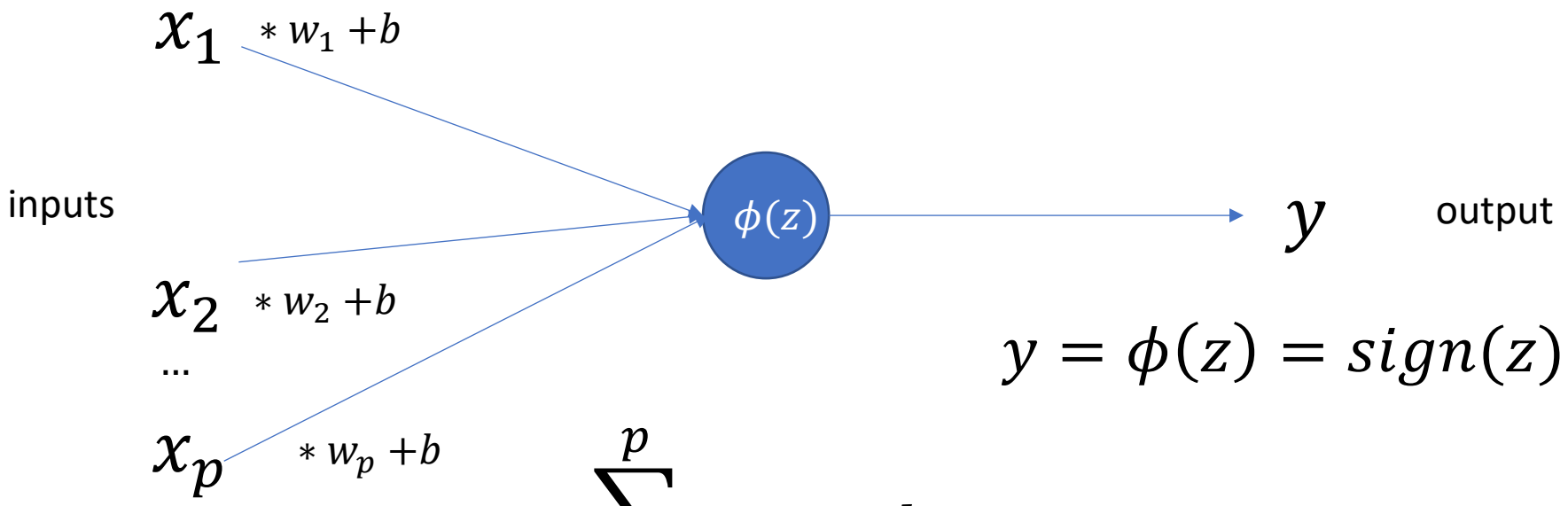


- Predict $y = \begin{cases} Green & if\ z \leq 0 \\ Red & if\ z > 0 \end{cases}$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Perceptron

- Let $w_1, w_2, \ldots, w_p$ be parameters for learning
- Let b be the bias to update w_i when x_i is zero



$$y = \phi(z) = sign(z)$$

$$z = \sum_{i=1}^{p} x_i w_i + b_i$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Perceptron Learning Algorithm

- Find weights using the Stochastic Gradient Descent methodology
  - Start with random w values
  - Update the w values as follows

$$w(k+1) = w(k) + \eta(y_i - \hat{y}_i)(x_i)$$

  - η (eta) is called the Step Size
    - η may decrease with the number of iterations
- Iterate through each row and adjust w for each point misclassified
- Epoch: an arbitrary cutoff, generally defined as "one pass over the entire dataset", used to separate training into distinct phases, which is useful for logging and periodic evaluation.

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

- **Example**: Perceptron Stochastic Gradient Descent
- Given a loan applicant's business age as well as the number of days delinquent in payments, will that applicant default in a specific loan?
- Historical Data is shown as follows:

Business Age (Ave = 10, SD=3)	Number of Days Delinquent (Ave = 50, SD=10)	Default?
13 years	70 days	Yes
7 years	70 days	No
10 years	40 days	No

- Standardized Values:

- Standardized

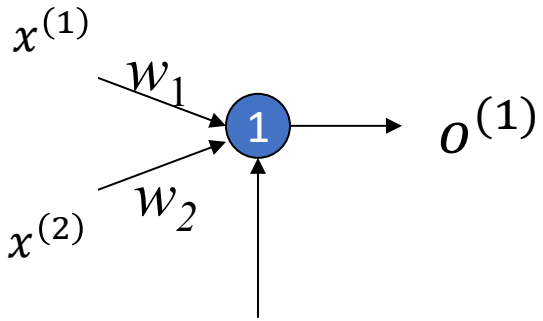
sBUSAGE	sDAYSDELQ	DEFAULT
1	2	Yes
-1	2	No
0	-1	No

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

- Network Design:

$$z^{(1)} = w_1 x^{(1)} + w_2 x^{(2)}$$

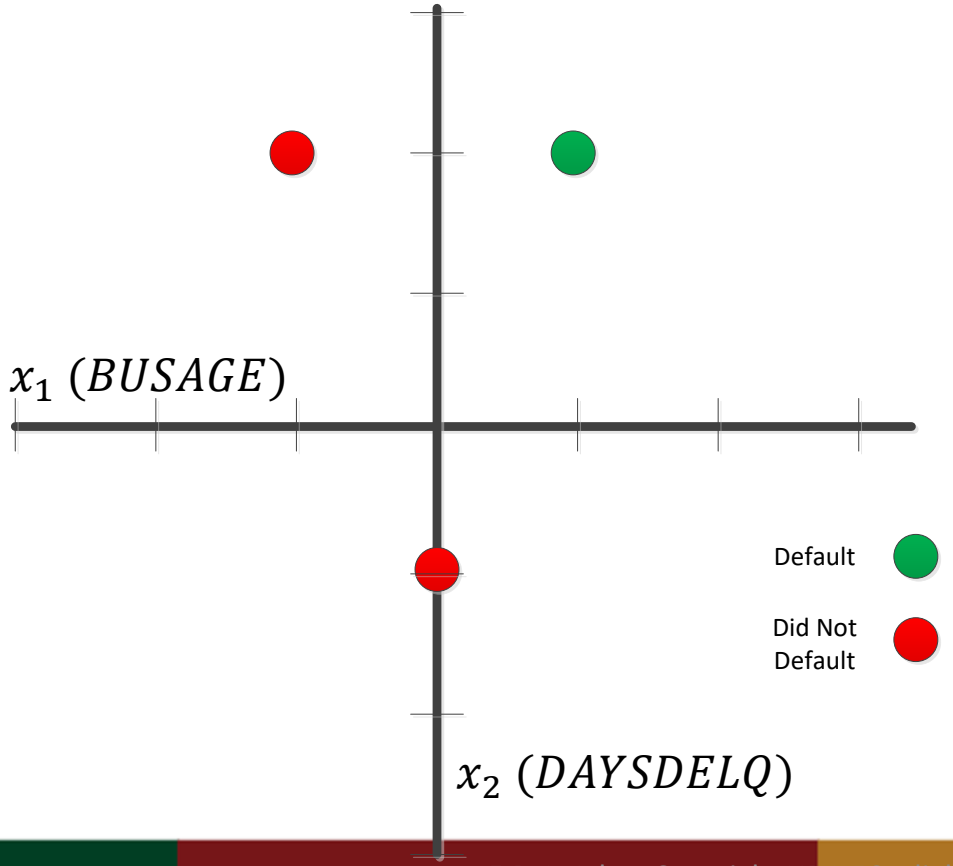


$$o^{(1)} = sign\left(z^{(1)}\right)$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

- Graphically:



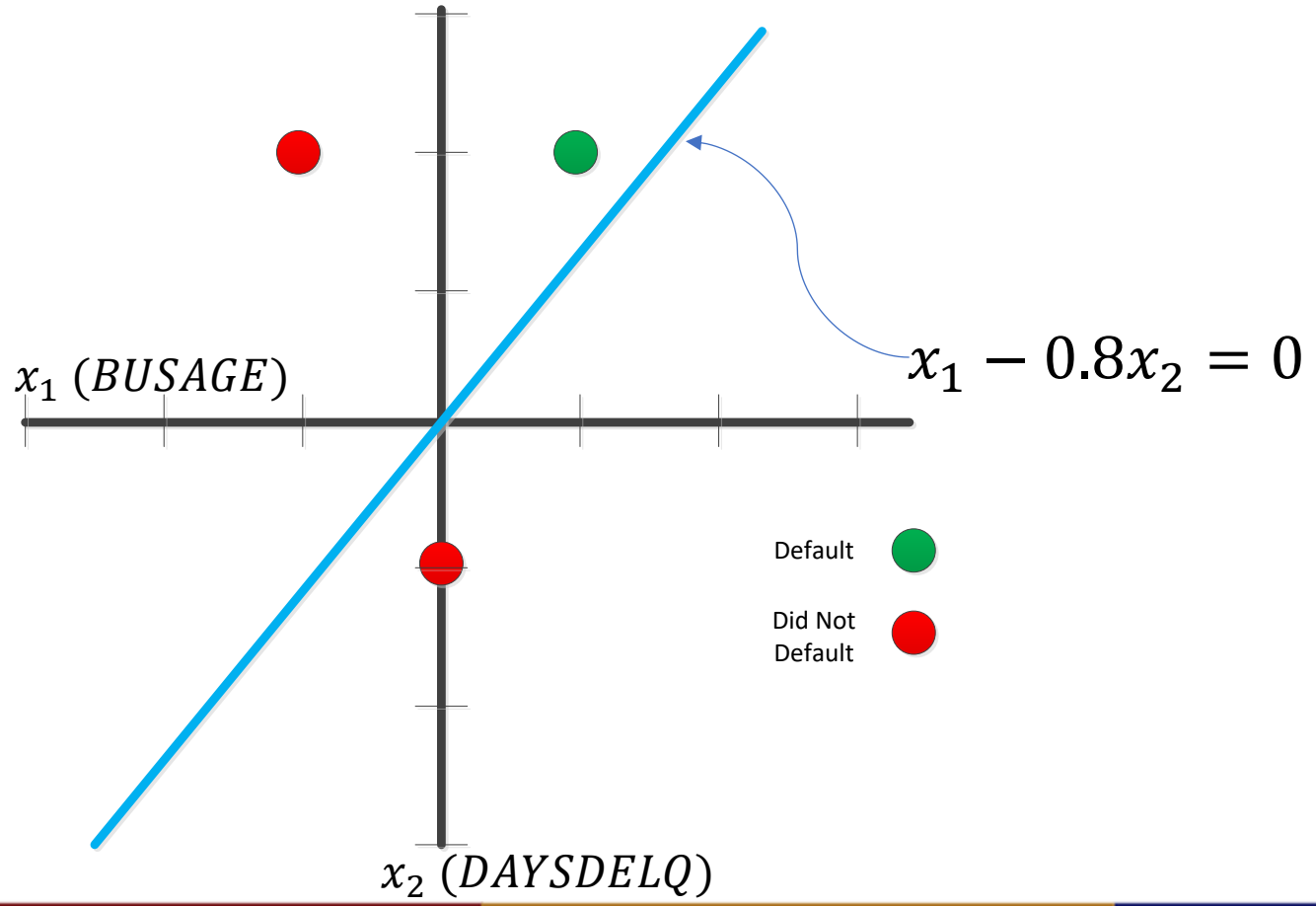
- Algebraically:

$$\left\{x = \begin{bmatrix}1\\2\end{bmatrix}, y = 1\right\}$$
$$\left\{x = \begin{bmatrix}-1\\2\end{bmatrix}, y = -1\right\}$$
$$\left\{x = \begin{bmatrix}0\\-1\end{bmatrix}, y = -1\right\}$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

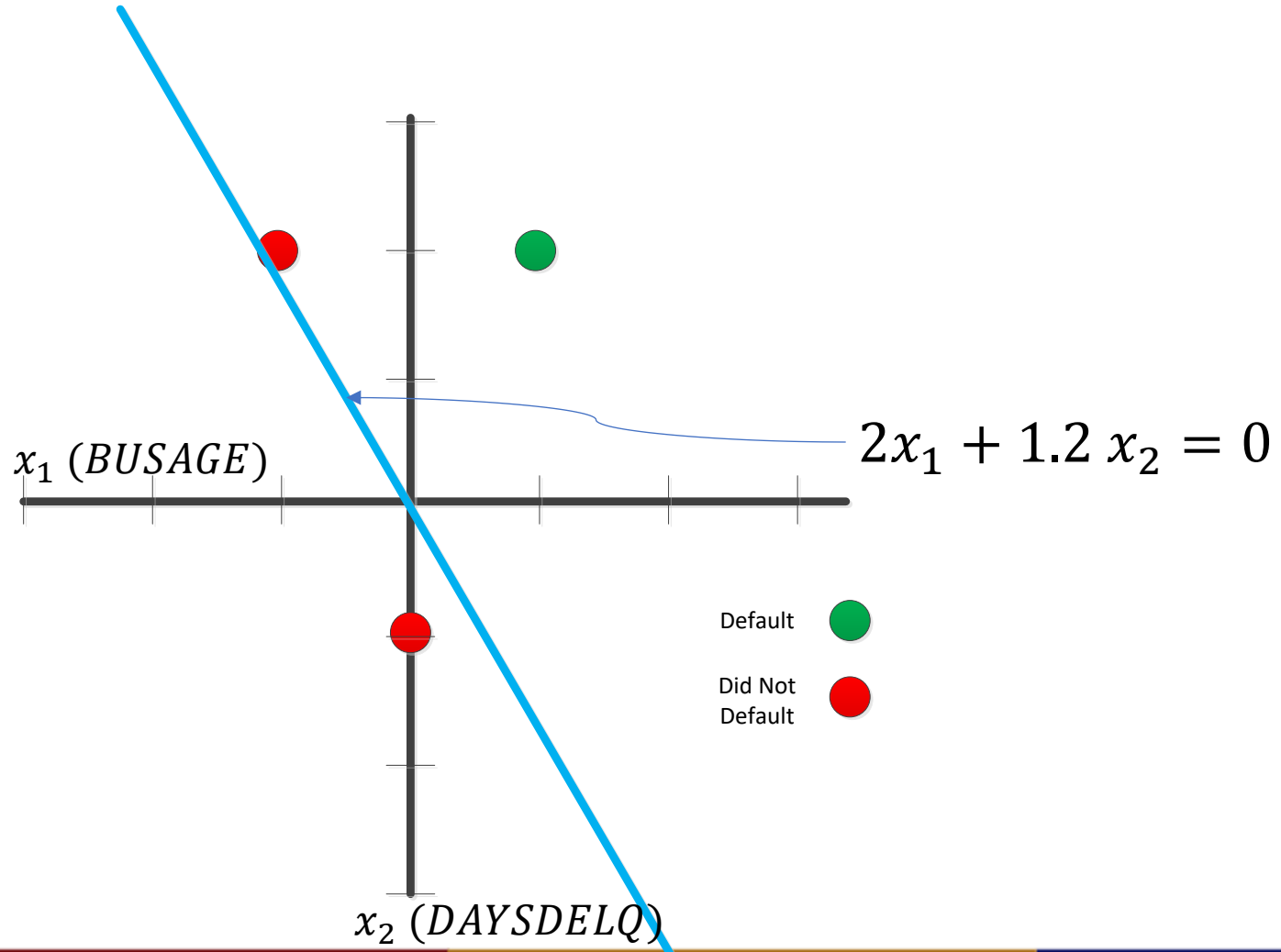
- Graphically:



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

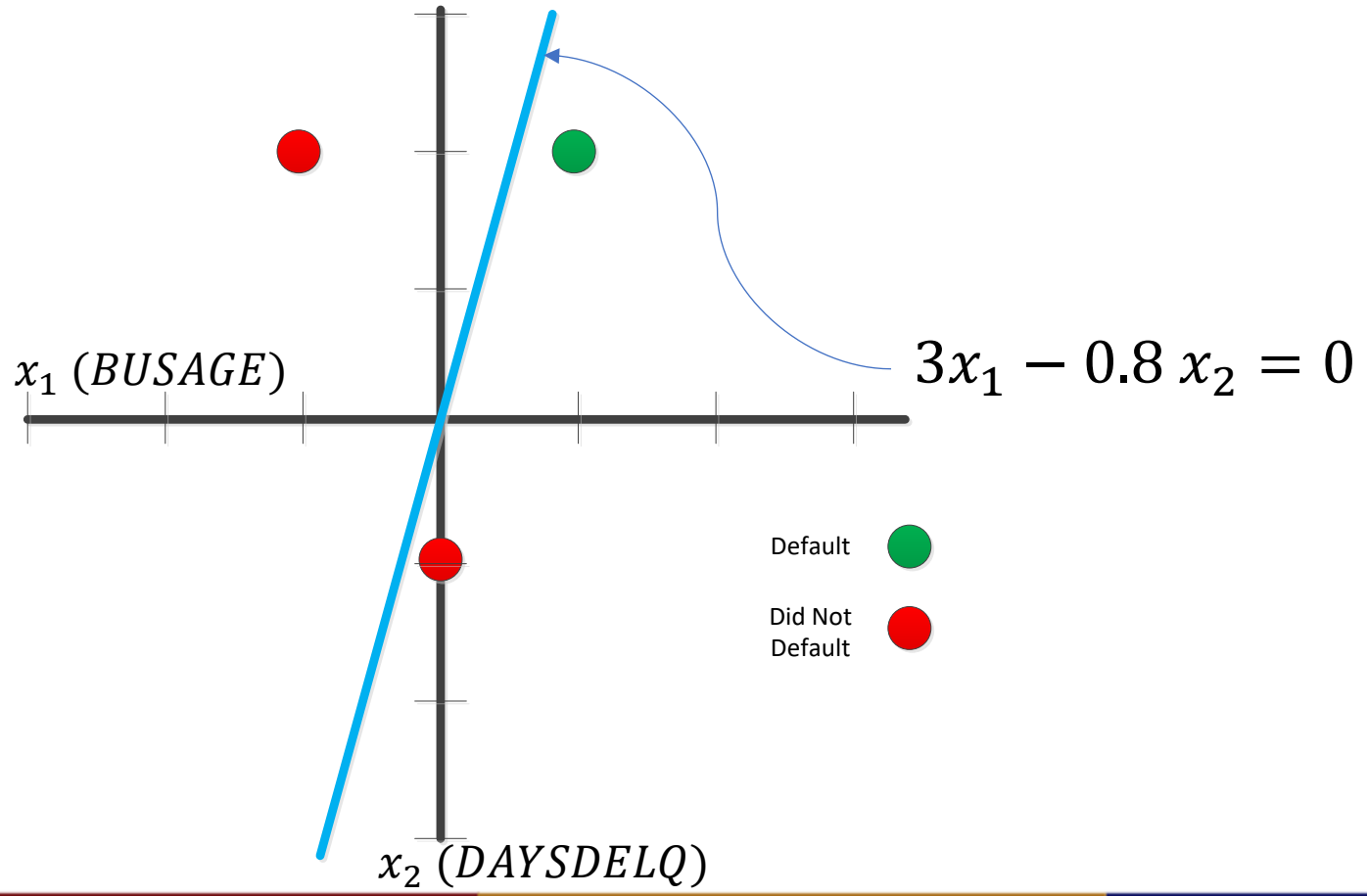
- Graphically:



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

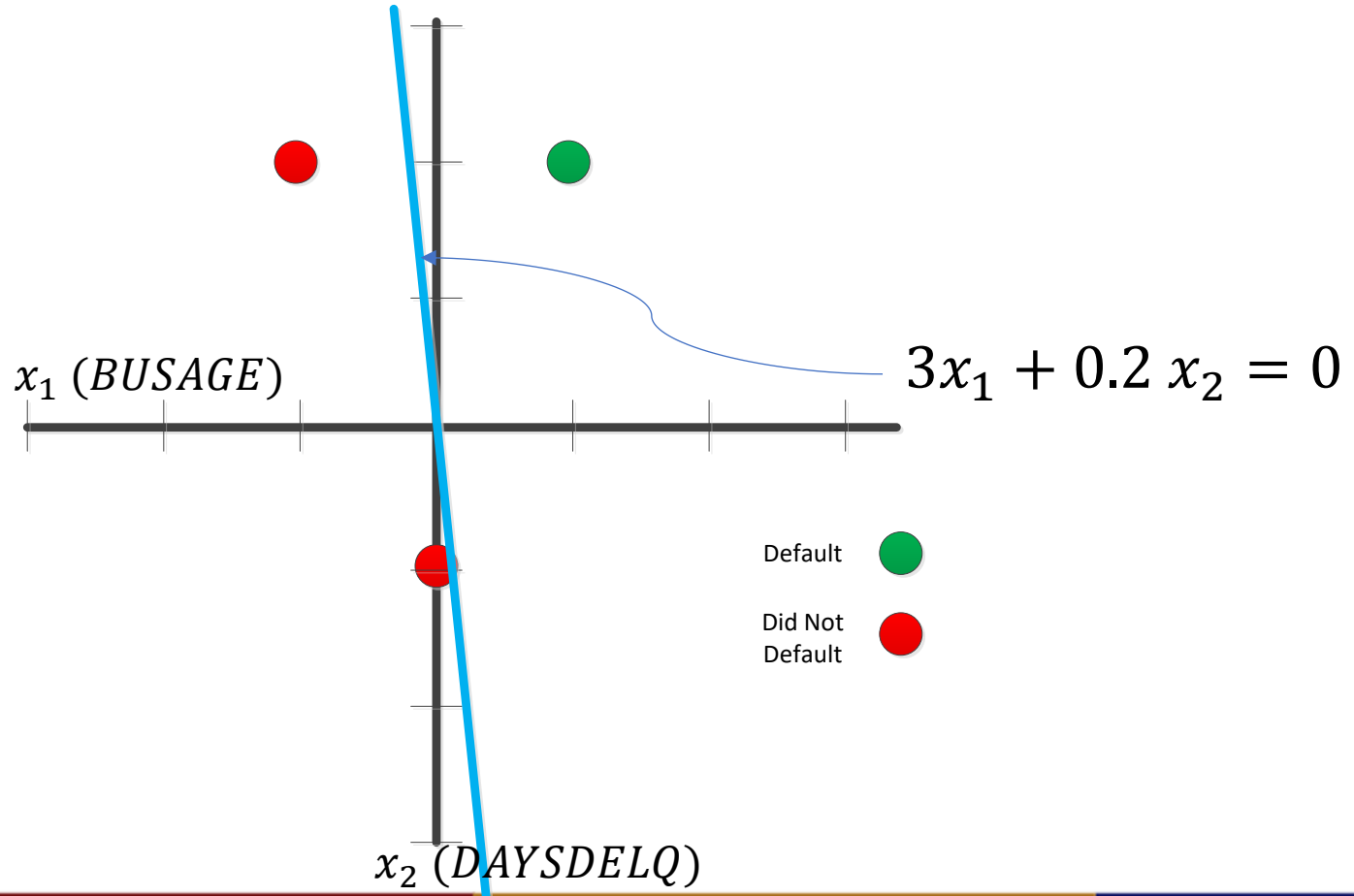
- Graphically:



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Simple Perceptron Induction Example

- Graphically:



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Perceptron Induction Usage

- Final Equation of the line:

$$3x_1 + 0.2\,x_2 = 0$$

BUSAGE	DAYSDELQ	DEFAULT
1	2	Yes
-1	2	No
0	-1	No

$$3(1) + 0.2(2) = 3.4 > 0$$

$$3(-1) + 0.2(2) = -2.6 < 0$$

$$3(0) + 0.2(-1) = -0.2 < 0$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Perceptron Induction Issues

- Separable data implies there is a plane that classifies training data perfectly
- Algorithm converges for separable data, if λ is sufficiently small
- If separable, many solutions, depends on the initial w
- If not separable, does not converge: Needs ANN

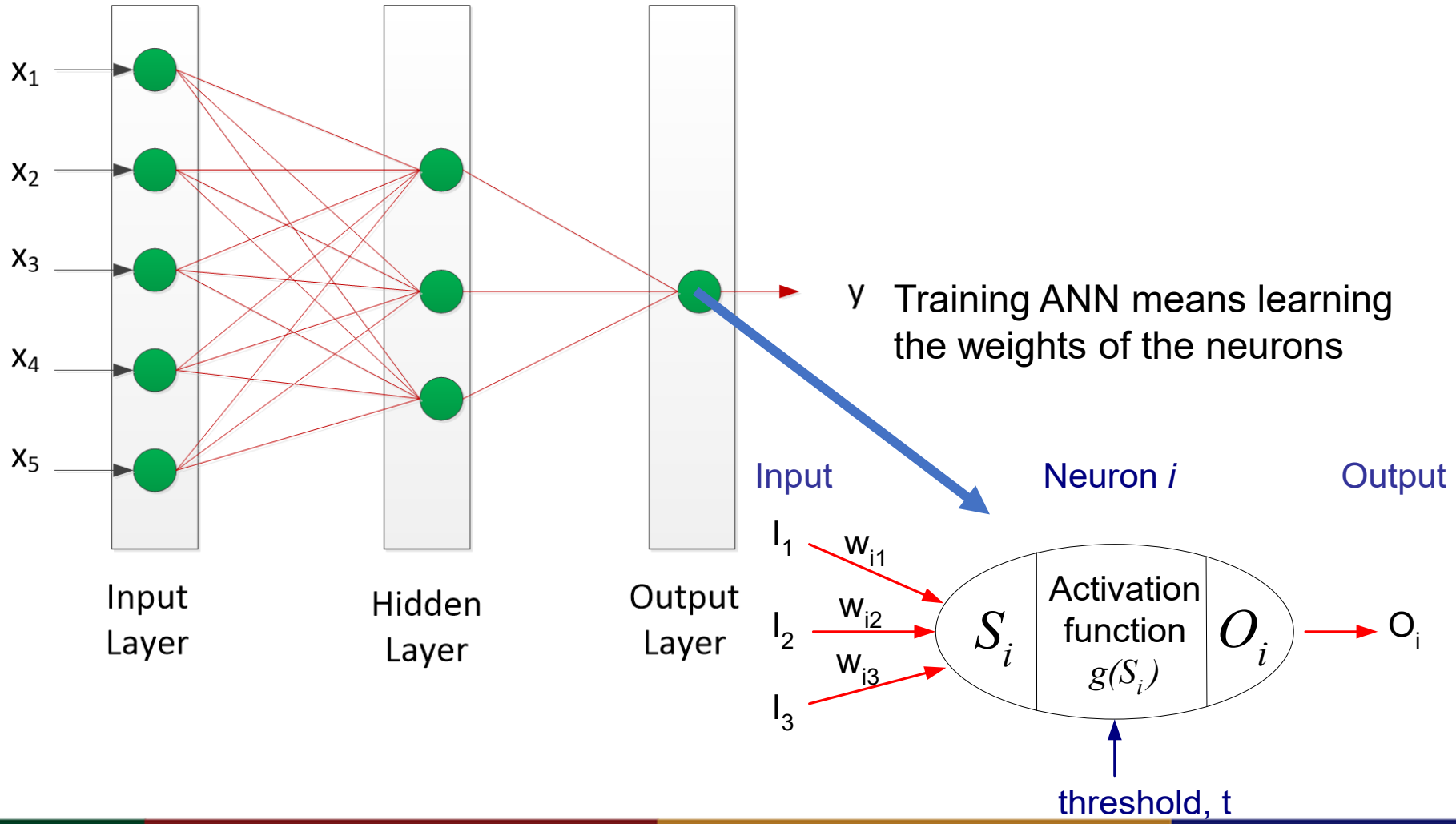
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Artificial Neural Networks

- **Definition: ANNs**
  - Is a **network of Perceptrons** or Nodes that mimics a biological network of Neurons in a brain
  - An ANN attempts to **recreate** the computational mirror of the biological neural network
  - Each neuron takes many **input signals**, then based on an internal **weighting system**, produces a single **output signal** that's typically sent as input to another **neuron**.
  - **Finding the weights** in an ANN constitute the bulk of the time done in learning the ANN.

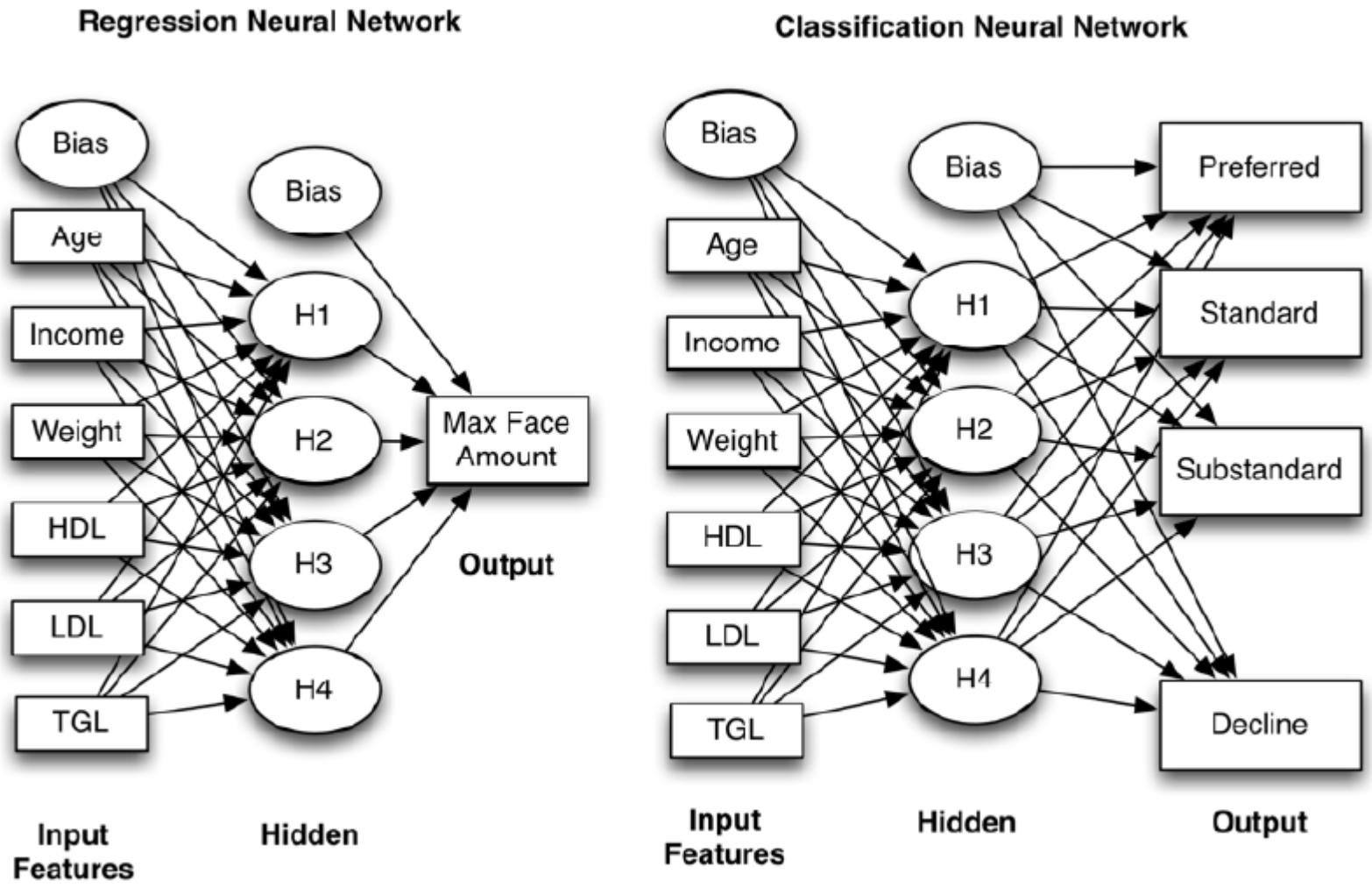
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

General Structure of ANN



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Artificial Neural Networks

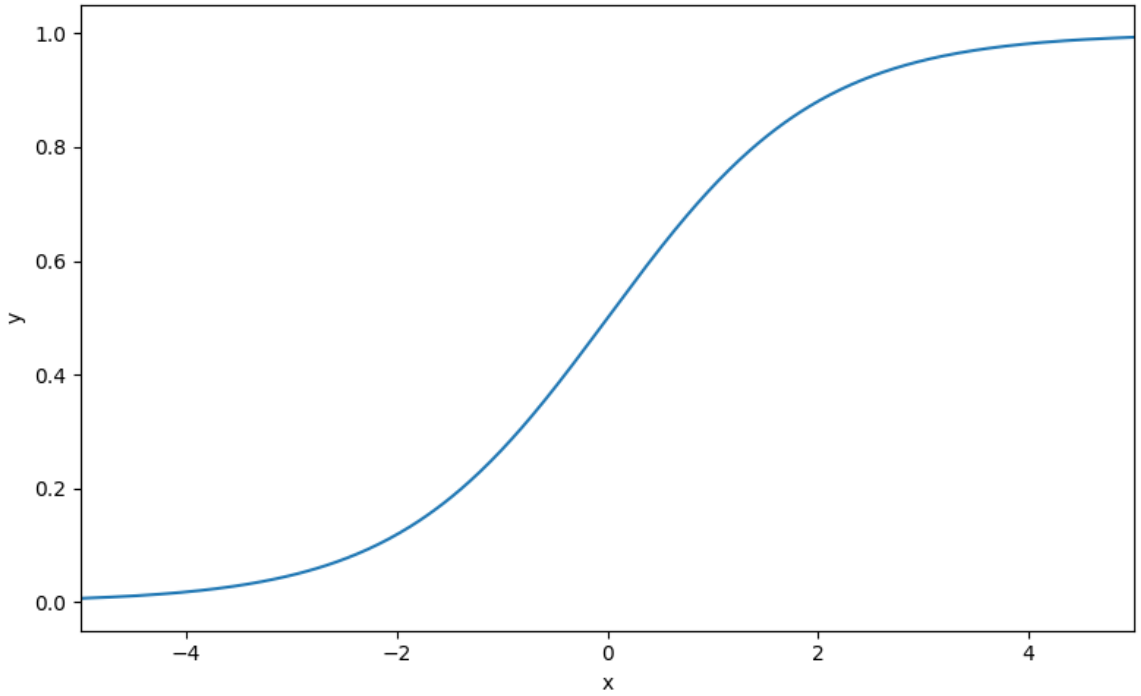


E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Activation Functions

- **Definition: Sigmoid Function**
  - The sigmoid or logistic activation function is a very common choice for feedforward neural networks that need to output **only positive numbers**.
  - The following equation shows the sigmoid activation function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Activation Functions

- **Definition: Softmax Function**
  - The softmax activation function forces the output of the neural network to represent the **probability** that the input falls into each of the classes.
  - Without the softmax, the neuron's outputs are simply numeric values, with the highest indicating the winning class.

$$\phi_i(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

- **Example**: [0.9, 0.2, 0.4]

$$sum = \exp(0.9) + \exp(0.2) + \exp(0.4) = 5.1728$$

$$\phi_i(z_1) = \frac{\exp(0.9)}{5.1728}, \phi_i(z_2) = \frac{\exp(0.2)}{5.1728}, \phi_i(z_3) = \frac{\exp(0.4)}{5.1728}$$

- Converted Probabilities:

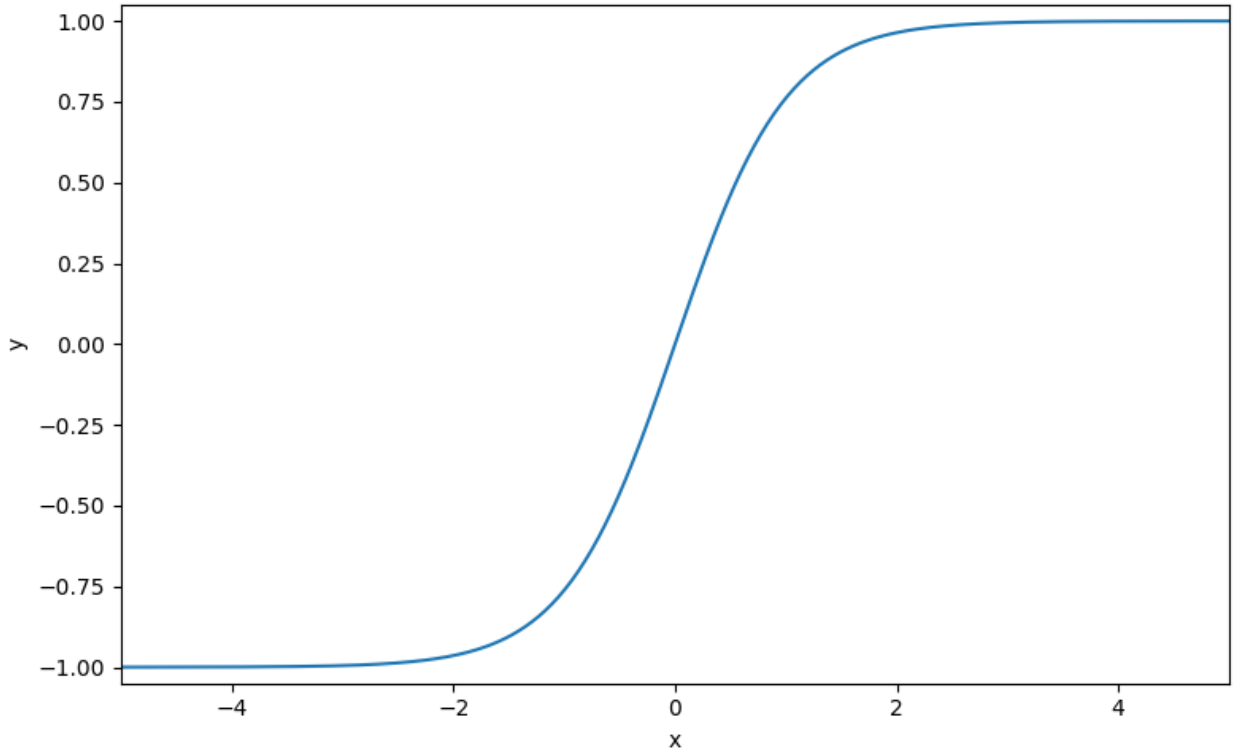
$$[0.4754, 0.2362, 0.2884]$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Activation Functions

- **Definition: TanH**
  - Tanh is the hyperbolic tangent mathematical function. It maps any real input number to an S-shaped output range between -1 and 1, centered at zero
  - Expressed via exponentials as:

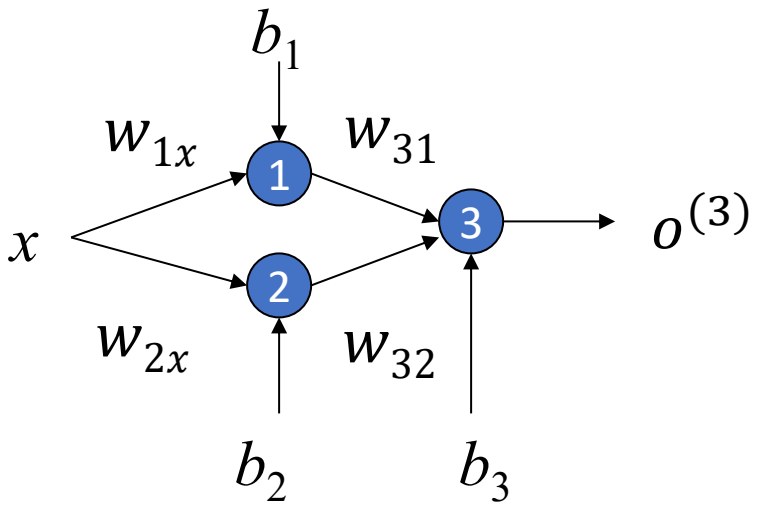
$$\tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}}$$



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Forward Propagation

- **Example: Hypothetical Data Example**
- Perform a forward pass on the following ANN $x = 1$



$w_{1x} = -0.27, w_{2x} = -0.41$

$b_1 = -0.48\,, b_2 = -0.13$

$w_{31} = 0.09, w_{32} = -0.17$

$b_3 = 0.48$

- For the hidden layer, ϕ=sigmoid; for the output layer, ϕ =linear
- Output Actual Value for $x = 1 \Rightarrow y = 0.5$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Forward Propagation

- Calculate the output for $x = 1$

$$z^{(1)} = w_{1x}x^{(1)} + b_1 = -0.27(1) - 0.48$$

$$z^{(1)} = -0.75$$

$$o^{(1)} = \sigma\left(z^{(1)}\right) = \frac{1}{1 + e^{-z^{(1)}}} = \frac{1}{1 + e^{0.75}}$$

$$o^{(1)} = 0.321$$

$$z^{(2)} = w_{2x}x^{(1)} + b_2 = -0.41(1) - 0.13$$

$$z^{(2)} = -0.54$$

$$o^{(2)} = \sigma\left(z^{(2)}\right) = \frac{1}{1 + e^{-z^{(2)}}} = \frac{1}{1 + e^{0.54}}$$

$$o^{(2)} = 0.368$$

$$z^{(3)} = w_{31}o^{(1)} + w_{32}o^{(2)} + b_3$$

$$z^{(3)} = 0.09(0.321) - 0.17(0.368) + 0.48$$

$$o^{(3)} = lin\left(z^{(2)}\right) = lin(0.446) = 0.446 = \hat{y}$$

$$e = y - \hat{y} = 0.5 - 0.446 = 0.054$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Loss or Cost Functions

- A loss function, or cost function, is a function around our model's predict function that tells us **"how good"** the model is at making predictions for a given set of parameters.
- The loss function has its own curve and its own **derivatives**.
- The Cost Function (Loss Function) is a measure of the difference between the **actual y value** to the **predicted $\hat{y}$ value**

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Cost Functions

- Let $o^{(L)}$ be the activation output of the L layer and n be the number of training points
- **Definition: Mean Squared Error**, or $L2$ loss is given by:

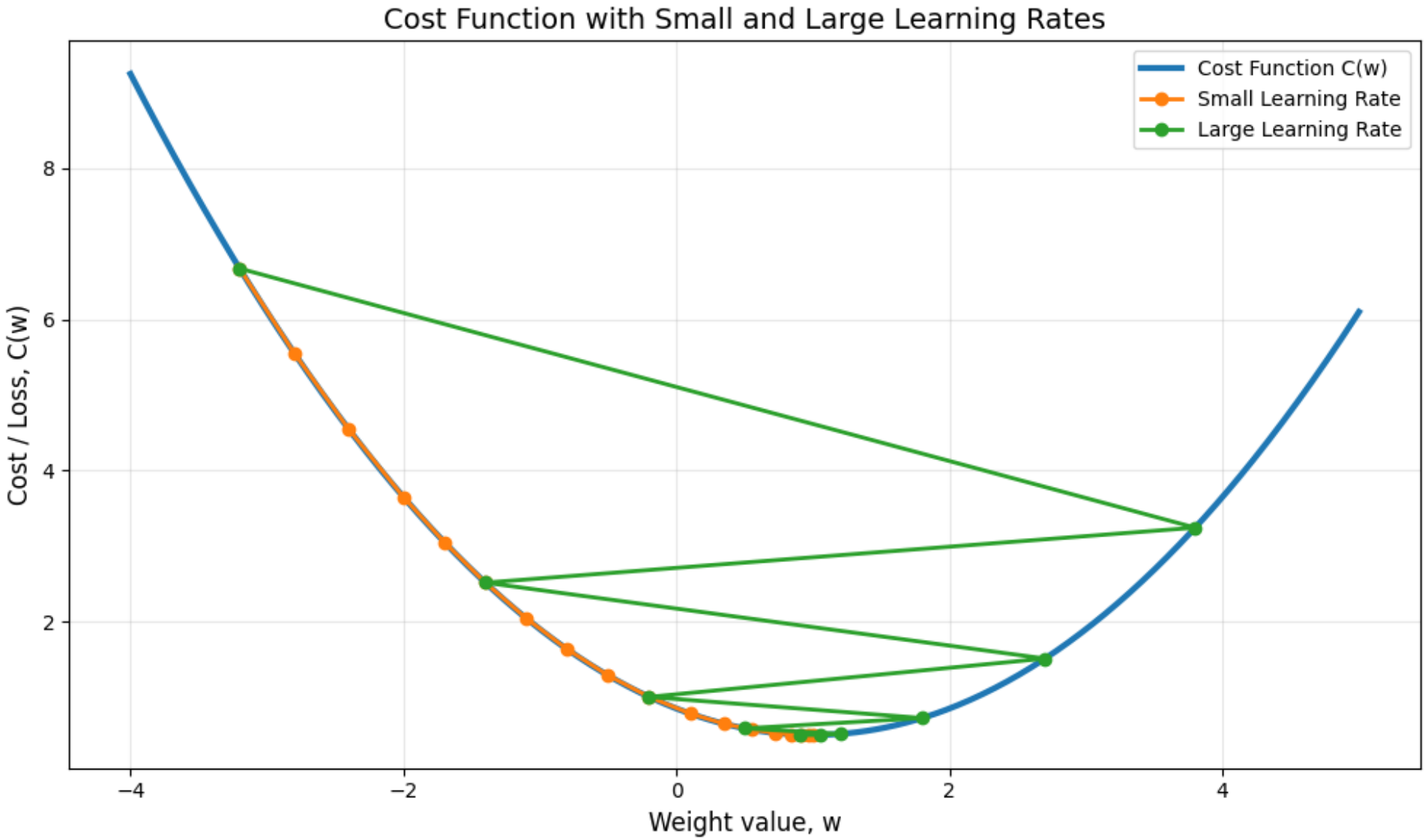
$$C(w_1, w_2 \ldots) = \frac{1}{2n}\sum_x \left(y_i - o^{(L)}\right)^2$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Training Neural Networks

- Backpropagation is one of the most common methods for **training a neural network**.
  - Rumelhart, Hinton, & Williams (1986) introduced backpropagation, and it remains popular today.
- Modelers frequently train deep neural networks with backpropagation because it scales really well when run on **graphical processing units** (GPUs).
- Classic backpropagation has been extended and modified to give rise to many different training algorithms.

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

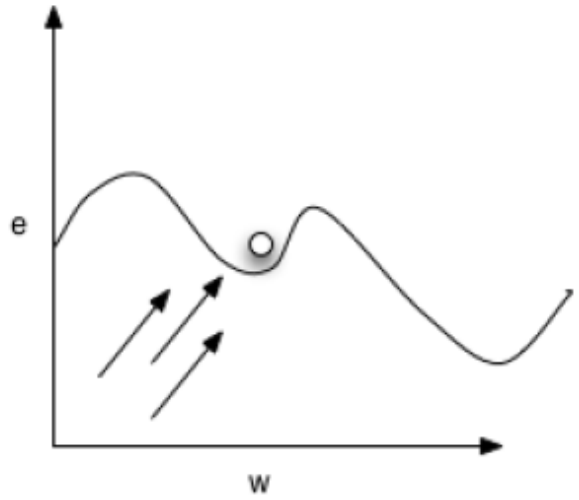
Momentum

- Momentum λ adds another term to the calculation of Δw

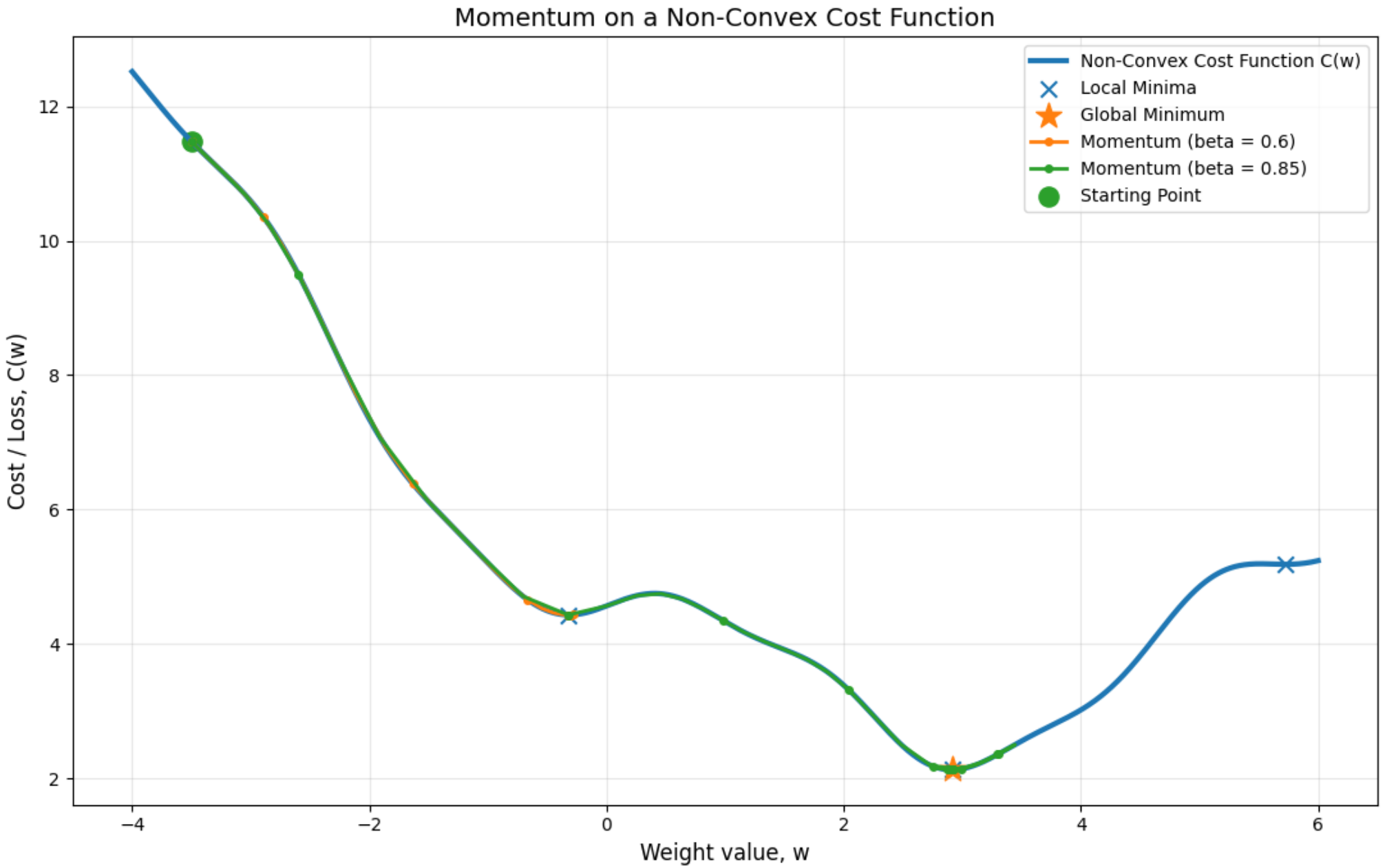
- This has the effect of adding **additional force** behind a direction a weight was moving

$$\Delta w(t) = \eta \nabla C(w) = \eta \frac{\partial C(w)}{\partial w_{ij}} + \lambda \Delta w(t-1)$$

- A very common value for momentum is 0.9.



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Other Parameters

- **Gradients** can be calculated for a training set element.
- These gradients can also be summed together into batches and the weights updated once per batch.
  - Online Training - Update the weights based on gradients calculated from a single training set element.
  - Batch Training - Update the weights based on the sum of the gradients over all training set elements.
  - Batch Size - Update the weights based on the sum of some batch size of training set elements.
  - Mini-Batch Training - The same as batch size, but with a very small batch size. Mini-batches are very popular and they are often in the 32-64 element range.

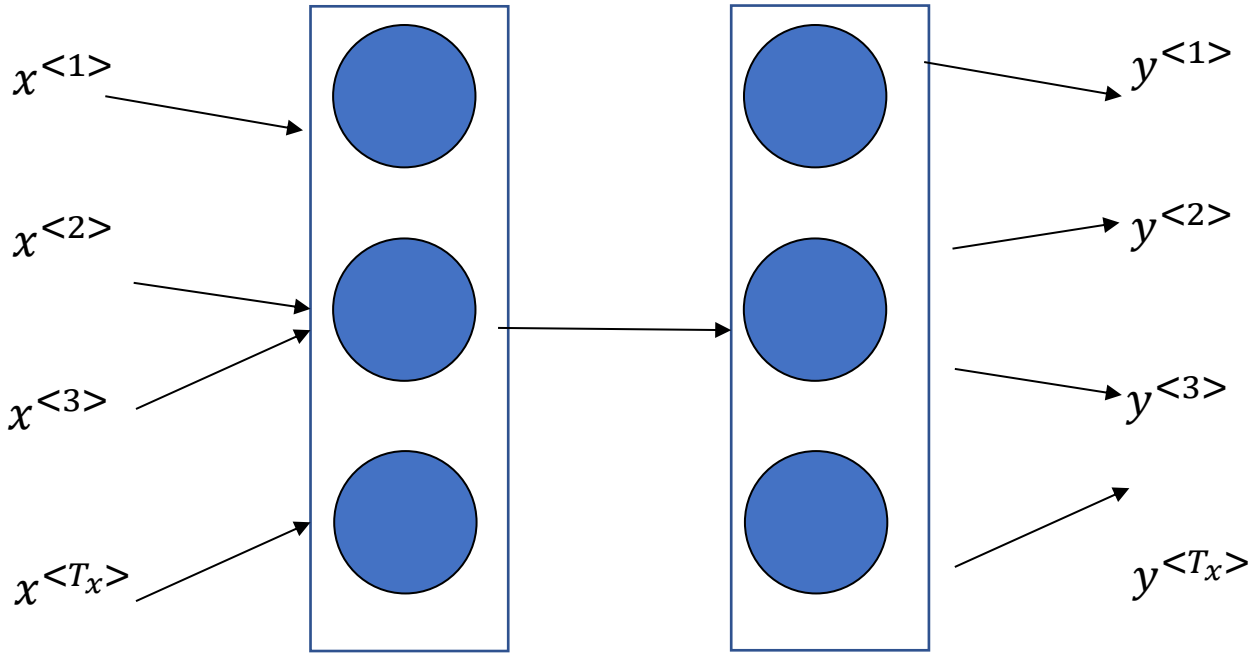
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Sequence Models

Problem	Predictor	Response
Speech Recognition	Audio File	“You're nothing, but a second-rate trying hard copycat”
Music Generation	Any Integer	Music
Sentiment Classification	“I love you 3000”	Positive Sentiment
Machine Translation	“Voulez-vous chanter avec moi?”	“Do you want to sing with me?”
Video Activity Recognition		Running
Name Entity Recognition	I have met Tony Stark and Ned Stark.	I have met Tony Stark and Ned Stark.

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Why Not A Standard Network?

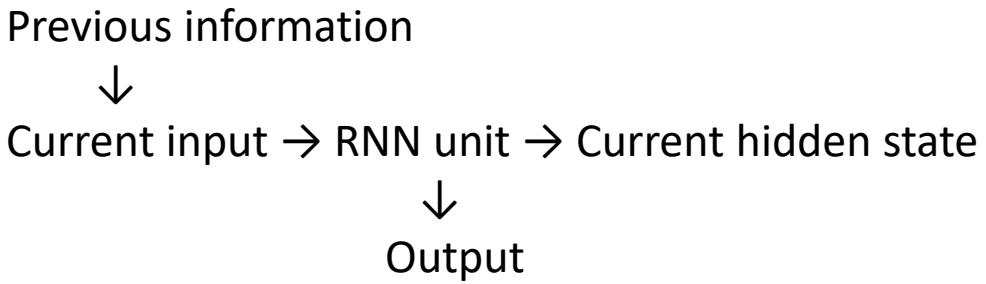


- Problems:
  - Inputs, Outputs can be different lengths in different examples
  - Does not share features learned across different position of text

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Why Not Use an Ordinary Neural Network?

- A conventional feed-forward neural network treats inputs as a fixed vector.
- It does not naturally maintain a memory of previous states.
- RNNs introduce recurrence:



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Training Neural Networks

- Backpropagation works by calculating a **weight change amount (Δw)** for every weight for iteration 0 $w_i(0)$ in the neural network.
- This value is subtracted from every weight by the following equation which will compute for the weight in iteration 1.

$$w(t) = w(t-1) - \Delta w(t-1)$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Δw

- The weight change is calculated depending on the training algorithm.
- Classic **backpropagation** simply calculates a gradient (∇, nabla) for every weight in the neural network with respect to the error function (C) of the neural network.

$$\Delta w(t) = \eta \nabla C(w) = \eta \frac{\partial C(w)}{\partial w_{ij}}$$

- The gradient is scaled by a learning rate eta (η) .
  - Too small of a learning rate will usually converge to a good solution; however, the process will be very slow.
  - Too large of a learning rate will either fail outright or converge to a higher error than a better learning rate.
  - Common values for learning rate are: 0.1, 0.01, 0.001

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Outline for this Session

- Introduction to ANNs
- **Transformers**
- Embeddings
- Attention
- Quantization
- Project Case

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com

Noam Shazeer* Google Brain noam@google.com

Niki Parmar* Google Research nikip@google.com

Jakob Uszkoreit* Google Research usz@google.com

Llion Jones* Google Research llion@google.com

Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu

Łukasz Kaiser* Google Brain lukaszkaiser@google.com

Illia Polosukhin* ‡ illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.0 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature.

Vaswani, Ashish, et al. "Attention is all you need." *Advances in neural information processing systems* 30 (2017).

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

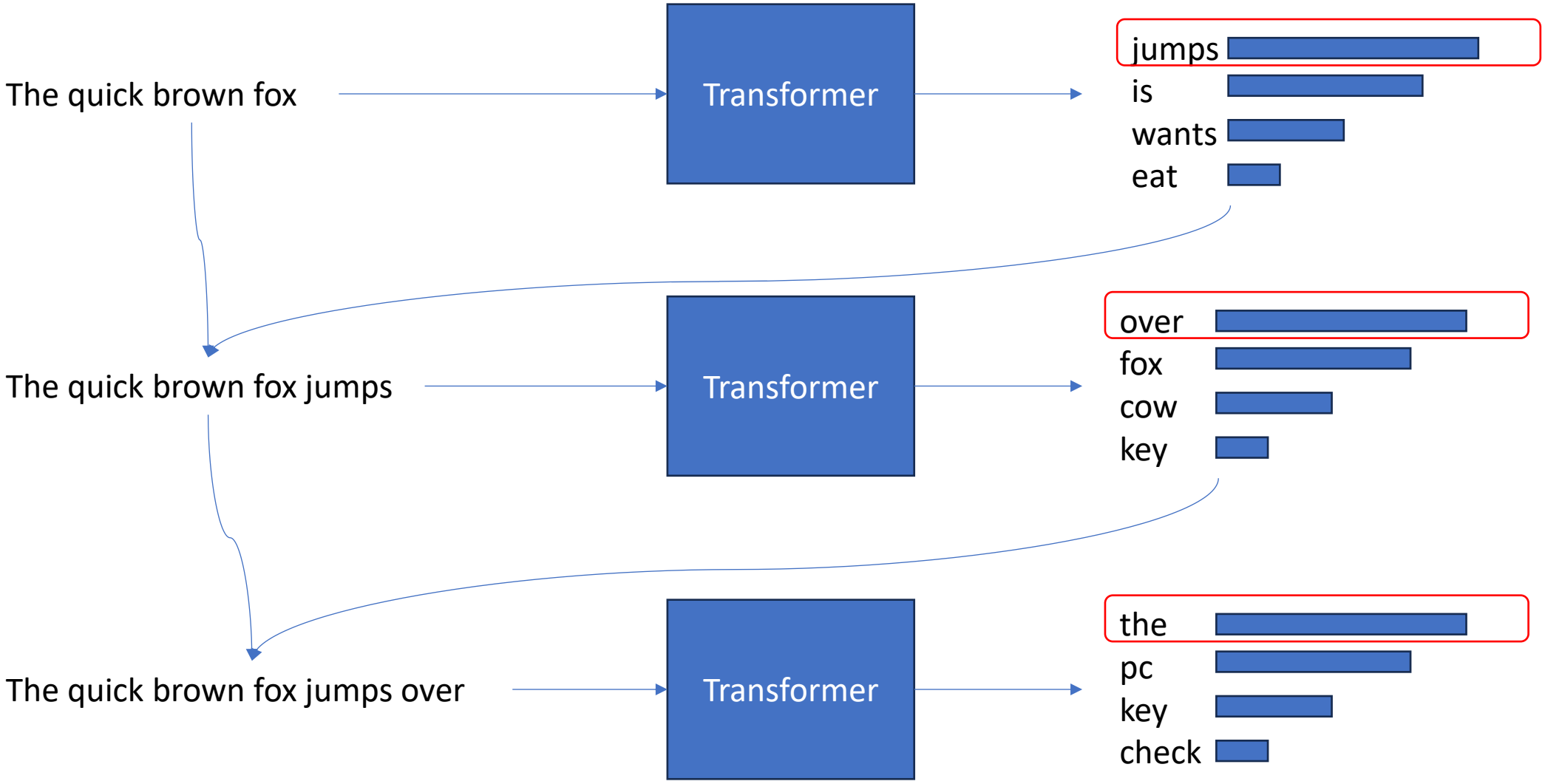
Transformer

- **Definition: A Transformer** is a breakthrough deep learning design introduced in the 2017 paper Attention Is All You Need by Vaswani et al.
- Allowed models like GPT (Generative Pre-trained Transformer) and BERT (Bidirectional Encoder Representations from Transformers) to process entire text sequences at once and understand deep contextual word relationships.



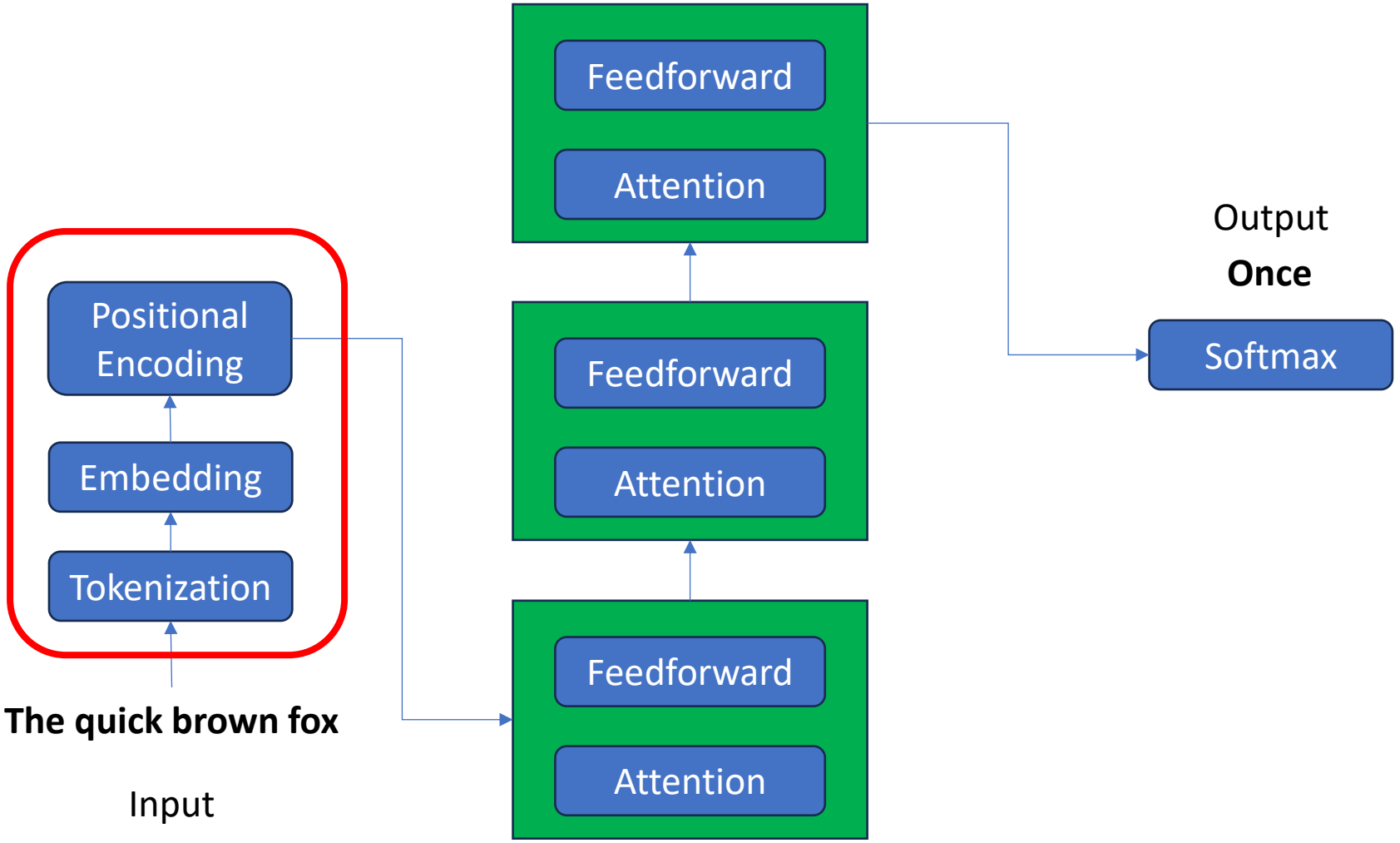
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Transformer



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Transformer Architecture



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Tokenization

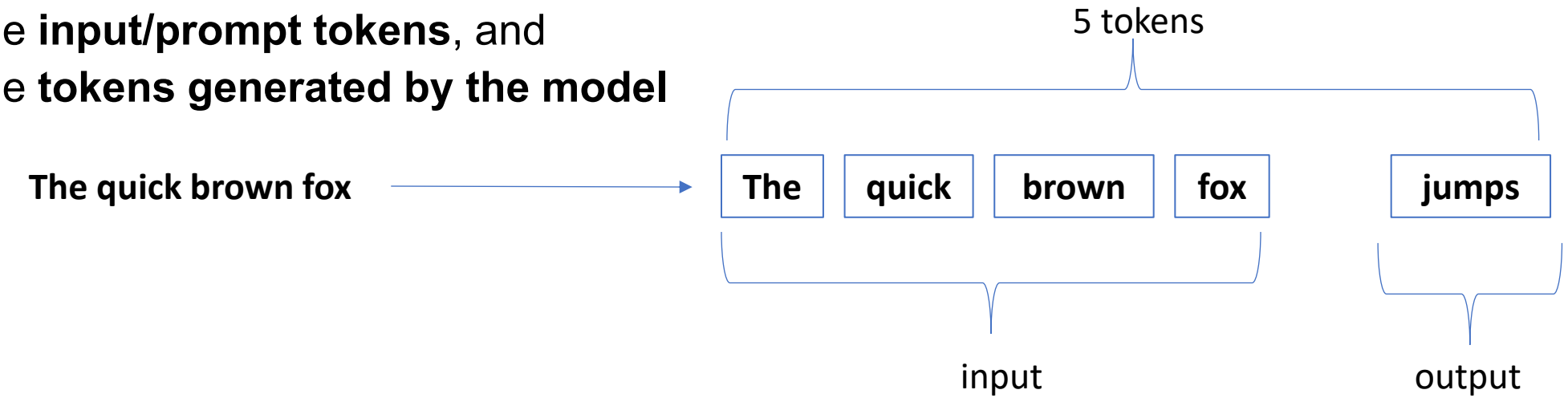
- **Definition: Tokenization** is the process of converting raw text into smaller units called **tokens** that can be mapped to numerical IDs and processed by the Transformer.
- **Methodology**
  - Input text is split into tokens such as words, subwords, characters, or punctuation.
  - Each token is mapped to a unique **token ID** from the model's vocabulary.
  - Special tokens may be added for tasks such as sequence start, end, padding, or separation.
  - The sequence of token IDs is passed to the **embedding layer**, which converts each ID into a dense vector.



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Tokenization

- **Definition: Context size** is the maximum number of tokens an LLM can consider at one time when processing or generating text.
- It includes both:
  - the **input/prompt tokens**, and
  - the **tokens generated by the model**



- GPT-3 has a context size of **2,048 tokens**
- If the prompt uses **1,500 tokens** then only about **548 tokens** remain available for generation before the context limit is reached.

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Outline for this Session

- Introduction to ANNs
- Transformers
- **Embeddings**
- Attention
- Quantization
- Project Case

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Embeddings

- **Definition: Embeddings** convert discrete token IDs into dense numerical vectors that represent information the Transformer can process.
- **Process:**
  - Each token ID is mapped to a learned vector of numbers.
  - Similar tokens can develop similar vector representations.
  - The embedding captures useful semantic and syntactic information.
  - Positional information is added so the model knows the order of tokens.
  - The resulting vectors become the input to the Transformer layers.



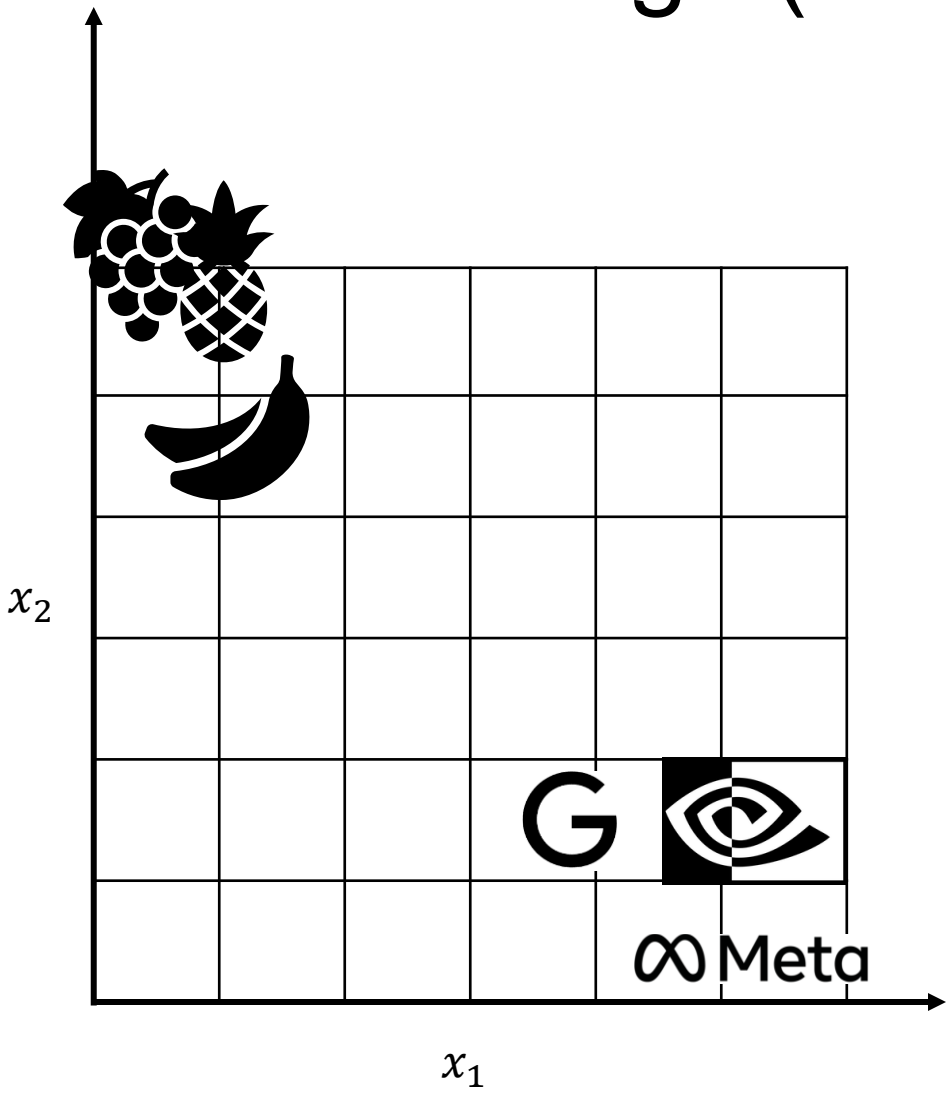
Neural Networks cannot interpret words



Neural Networks require numeric inputs

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Embeddings (Example 2 Dim Representation)

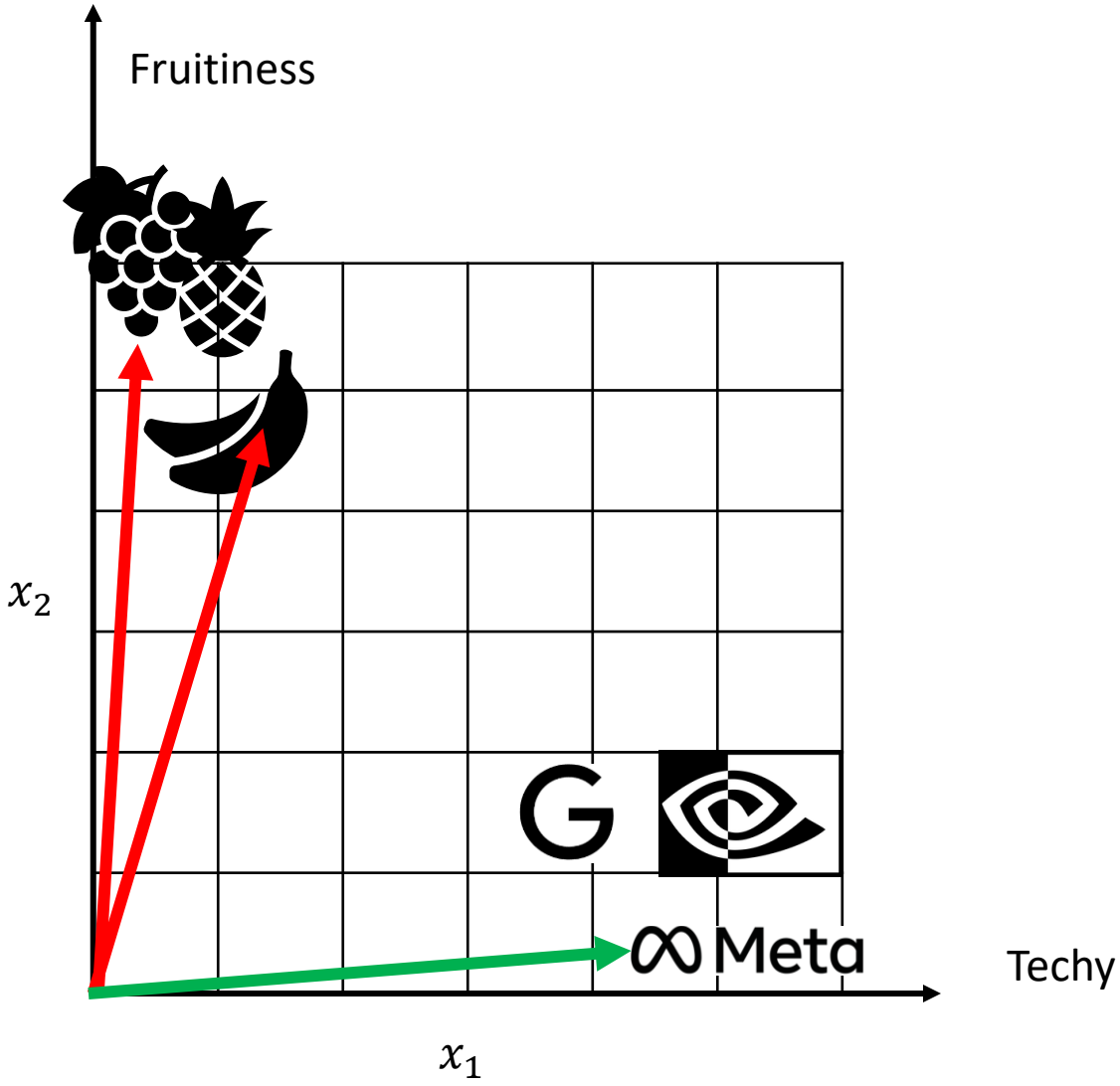


Word	x_1	x_2
Grapes	0.5	6
Pineapple	1	5.5
Banana	1	4.5
Google	3.75	1.5
Meta	5	0.5
Nvidia	5	1.5



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Dot Product



Word	x_1	x_2
Grapes	0.5	6
Banana	1	4.5

$$Similarity(Grapes, Banana) = 0.5 * 1 + 6 * 4.5 = 27.5$$

Word	x_1	x_2
Grapes	0.5	6
Meta	5	0.5

$$Similarity(Grapes, Meta) = 0.5 * 5 + 6 * 0.5 = 5.5$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Embeddings

50257 tokens

Embedding Matrix:

a	aah	aardvark	…	zygote	zymosis	zzzz
0.82	1.90	3.92	…	9.90	-1.90	-0.27
-1.09	1.90	0.27	…	-0.82	-1.09	-0.82
-2.17	-2.45	-1.36	…	0.27	-1.90	2.45
-1.63	-0.54	1.63	…	-1.36	0.82	2.72
⋮	⋮	⋮	⋱	⋮	⋮	⋮
1.63	0.00	-1.09	…	-1.09	-1.36	-1.36
2.45	-1.90	1.09	…	-2.17	0.54	-1.36

12,288 dim

Vocabulary:

GPT-3 175B Model: 50257 tokens

$$\left.\begin{matrix} a \\ aah \\ aardvark \\ aargh \\ \vdots \\ zygote \\ zymosis \\ zzz \end{matrix}\right\} All\ Words = \boldsymbol{d_{vocab}}$$

The **quick** **brown** **fox**

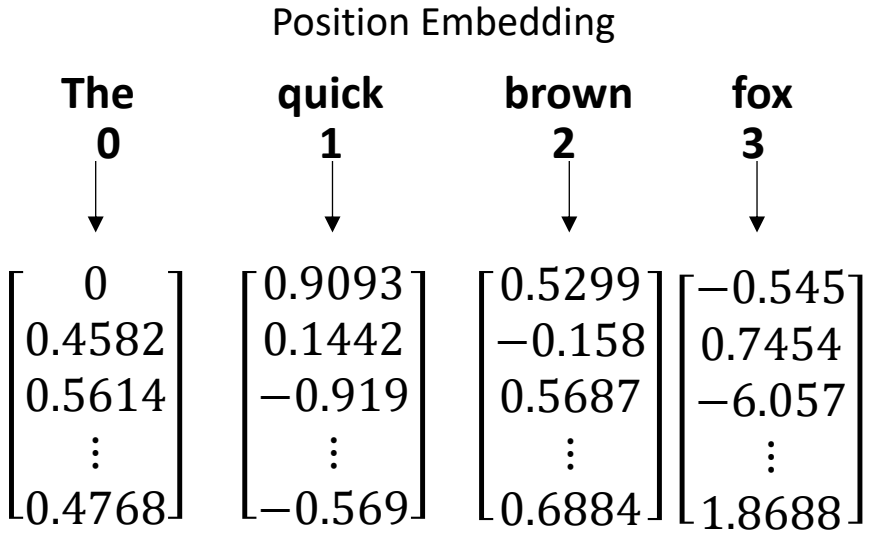
$$\begin{bmatrix} 1.56 \\ -3.69 \\ 2.04 \\ \vdots \\ 2.60 \end{bmatrix} \begin{bmatrix} 6.45 \\ 8.54 \\ 1.50 \\ \vdots \\ -2.65 \end{bmatrix} \begin{bmatrix} 1.56 \\ -3.69 \\ 2.04 \\ \vdots \\ 2.60 \end{bmatrix} \begin{bmatrix} -4.45 \\ 2.65 \\ -9.5 \\ \vdots \\ 5.64 \end{bmatrix}$$

$12{,}288 = \boldsymbol{d_{embed}}$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

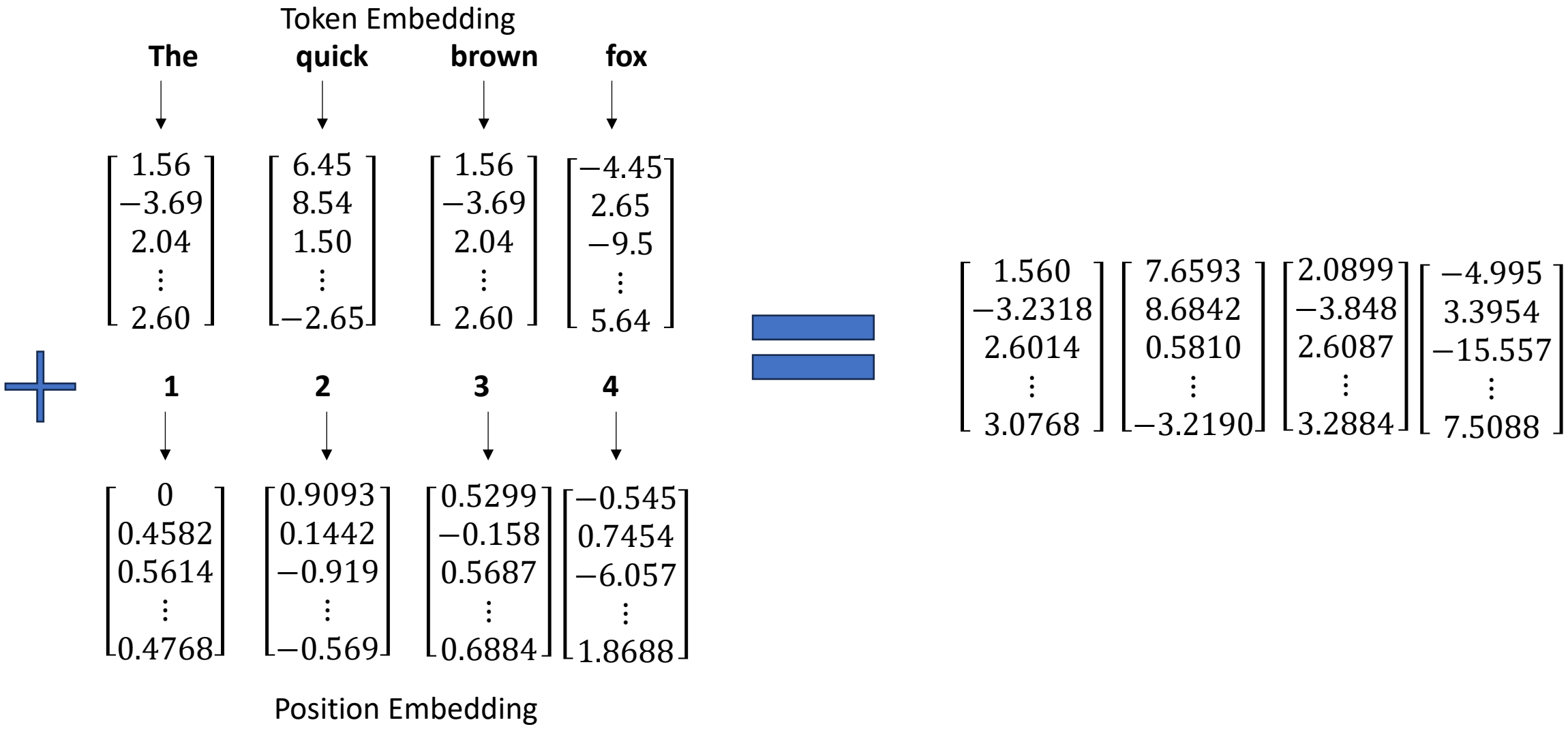
Position Encoding

- **Definition: Positional encoding** is the method used in a Transformer to provide information about the **order and position of tokens in a sequence**.
- Because self-attention processes tokens in parallel, the Transformer does not automatically know which token came first, second, third, and so on. Positional information is therefore added or incorporated into each token representation before or during attention.



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Position Encoding



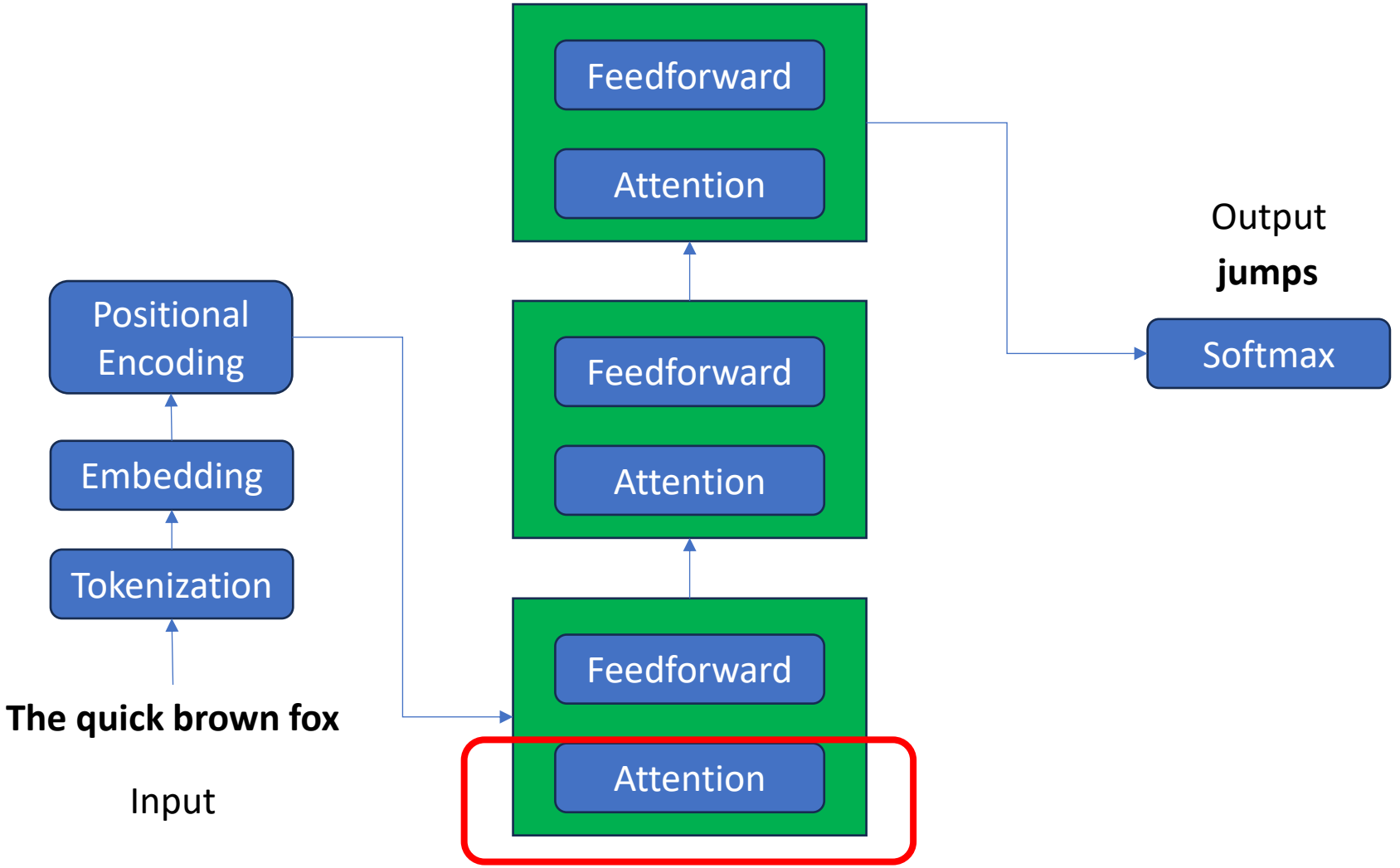
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

GPT-3 175B Parameter Count

Component	Formula	Computation
Embedding	$d_{embed} \times d_{vocab}$	$12{,}288 \times 50{,}257 = 617{,}558{,}016$
Key		
Query		
Value1		
Value2		
MLP Matrix 1		
MLP Matrix 2		
Unembedding		
	$Total$	

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Transformer Architecture



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

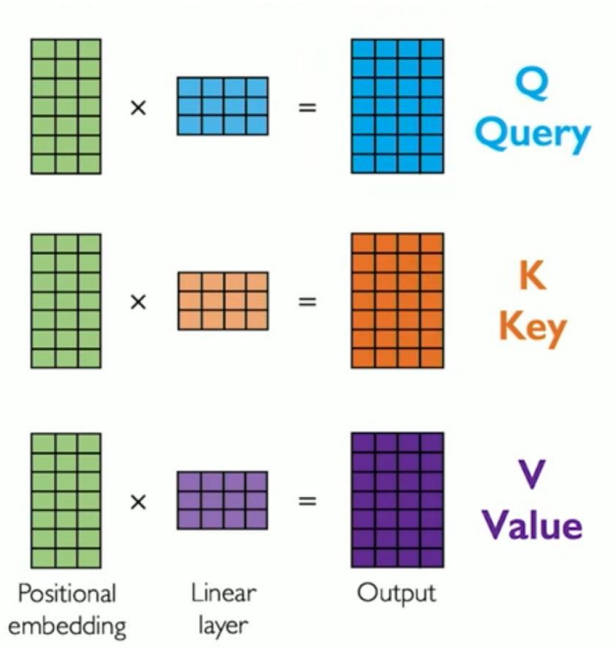
Outline for this Session

- Introduction to ANNs
- Transformers
- Embeddings
- **Attention**
- Quantization
- Project Case

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

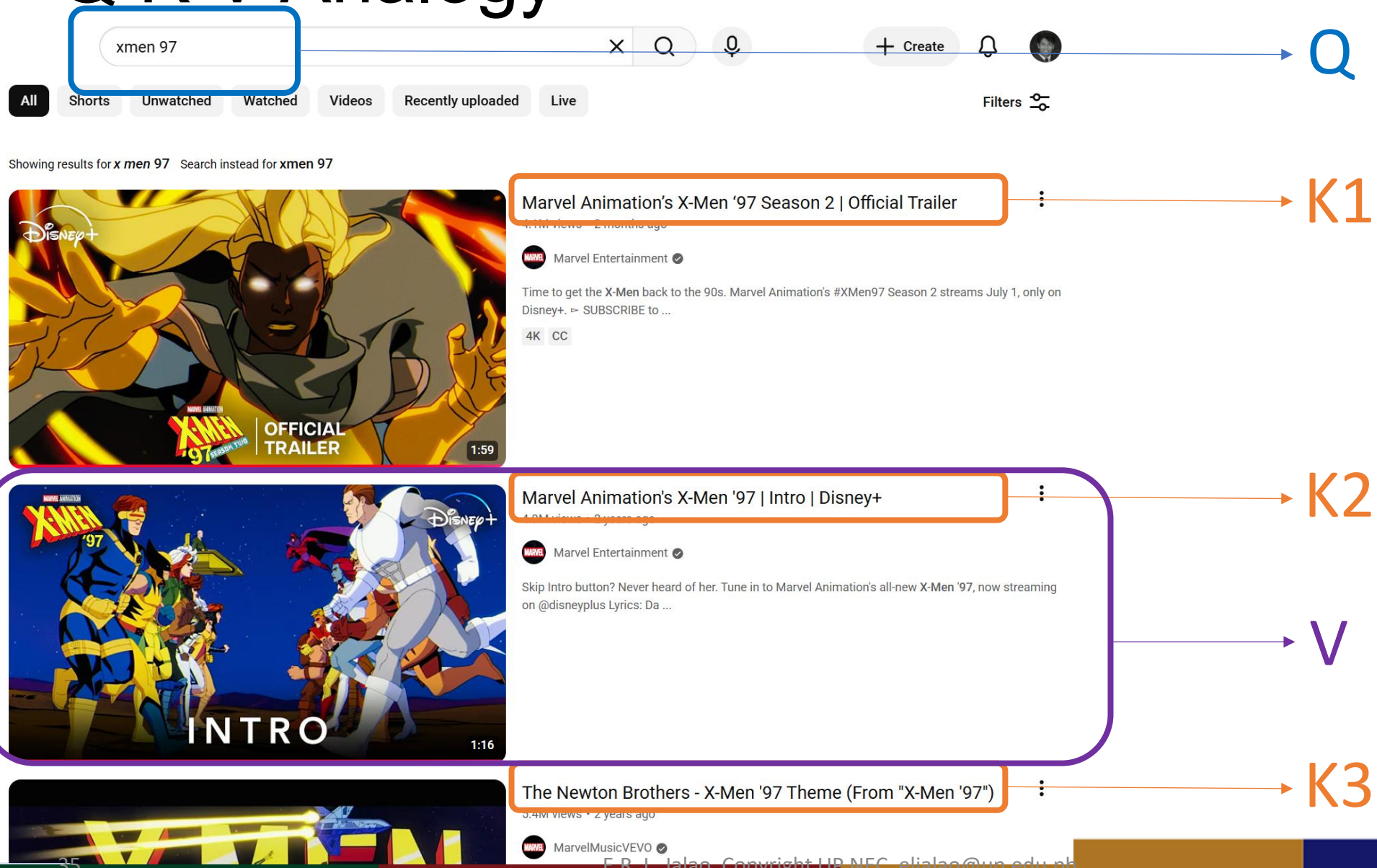
Attention

- **Definition: Attention** is the mechanism that allows a Transformer to determine which tokens in the input are most relevant to each other when building a contextual representation.
- Instead of processing each token independently, attention lets every token look at other tokens and assign them different levels of importance.
- For each token, the Transformer creates:
  - **Query (Q)** – what the token is looking for
  - **Key (K)** – what each token offers for matching
  - **Value (V)** – the information carried by each token



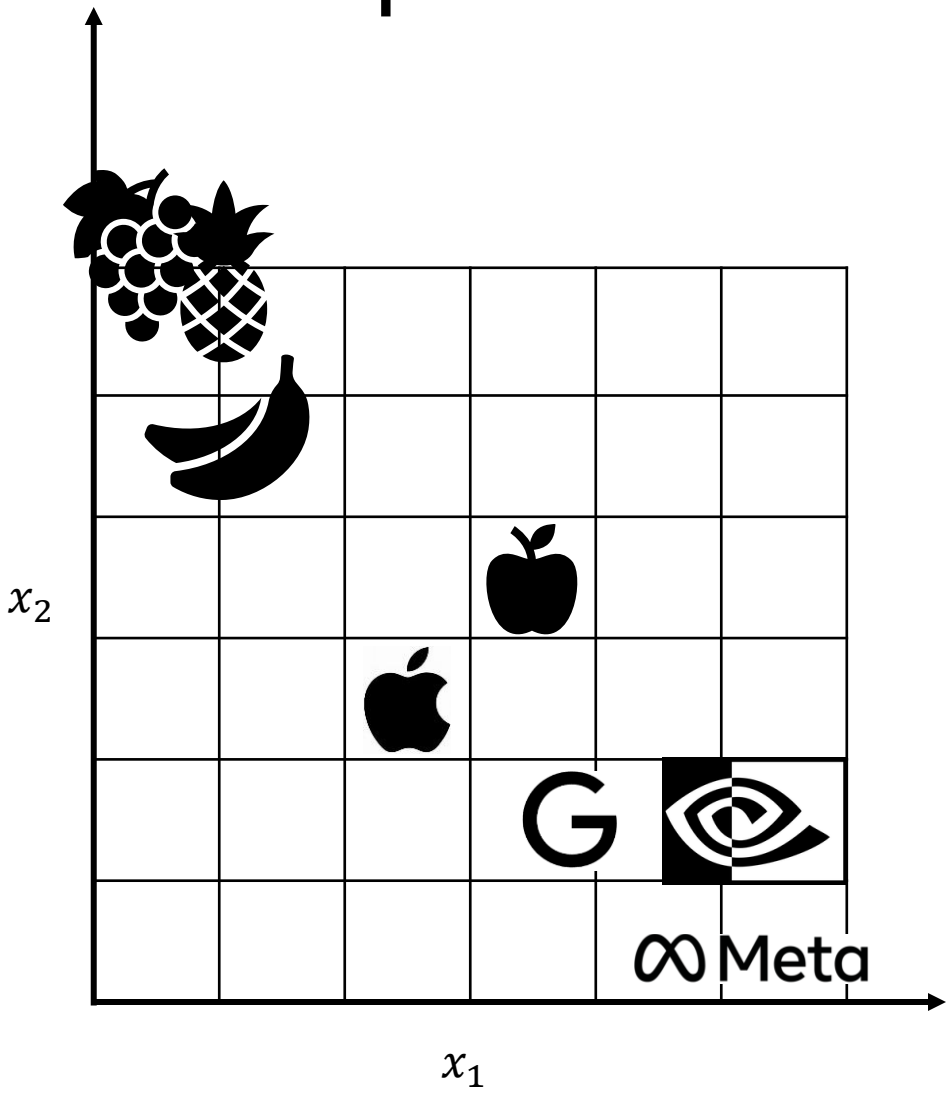
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Q K V Analogy



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Compute Attention Weighting

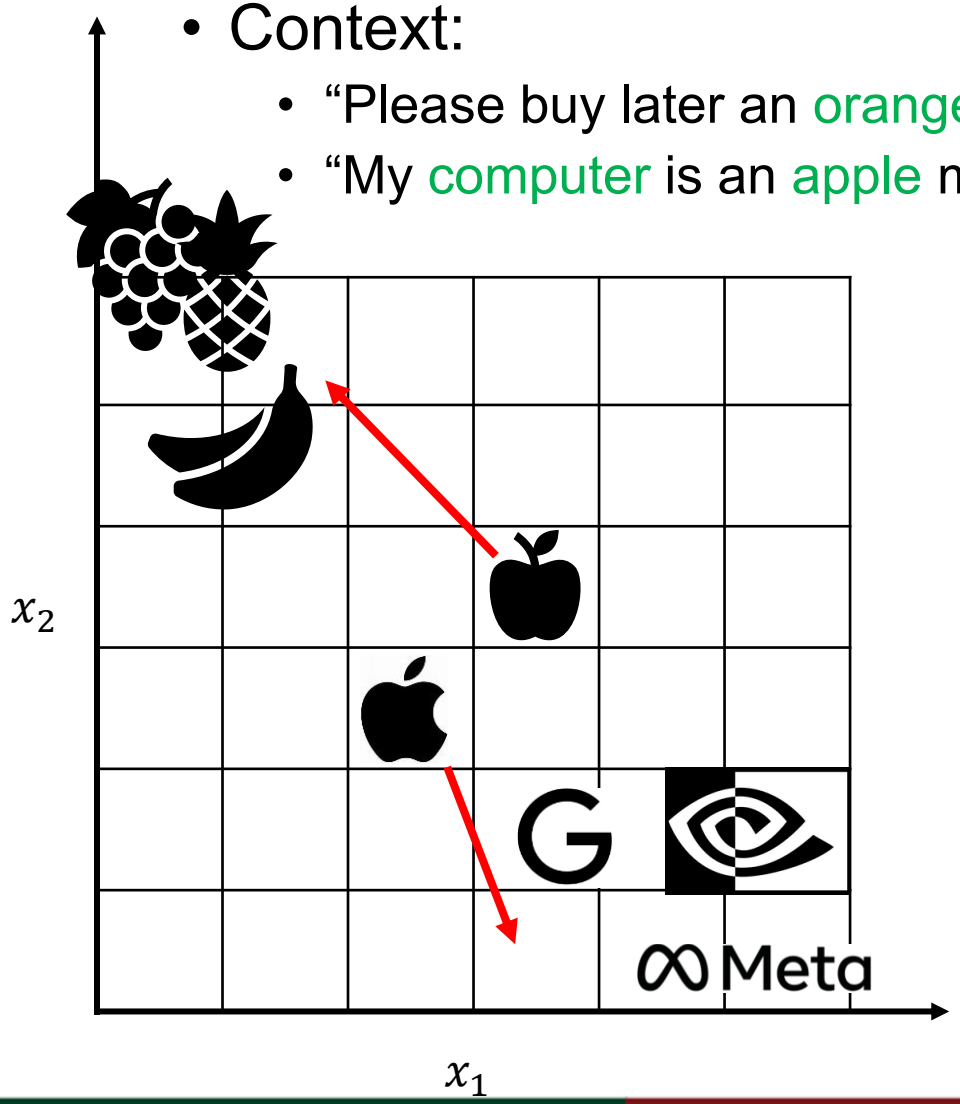


Word	x_1	x_2
Grapes	0.5	6
Pineapple	1	5.5
Banana	1	4.5
Google	3.75	1.5
Meta	5	0.5
Nvidia	5	1.5
Apple	4.5	4.5
Apple	3.5	3.5

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Compute Attention Weighting

- Context:
  - “Please buy later an orange and an apple.”
  - “My computer is an apple macintosh.”

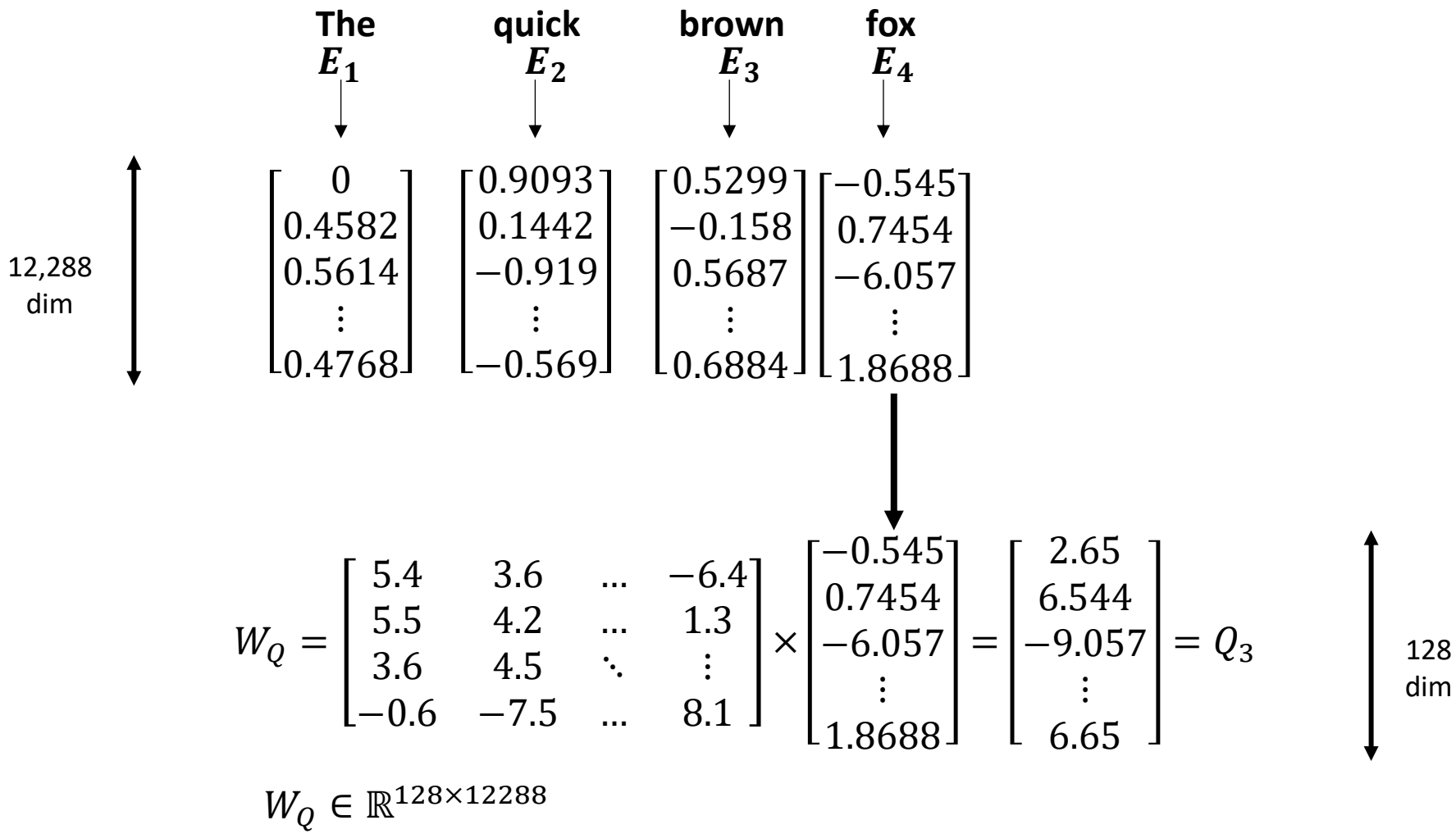


$$Attention(Q, K, V) = softmax\left(\frac{K^T Q}{\sqrt{d_k}}\right)V$$

Word	x_1	x_2
Grapes	0.5	6
Pineapple	1	5.5
Banana	1	4.5
Google	3.75	1.5
Meta	5	0.5
Nvidia	5	1.5
Apple	4.5-2	4.5+2
Apple	3.5+1	3.5-2

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Q Computation

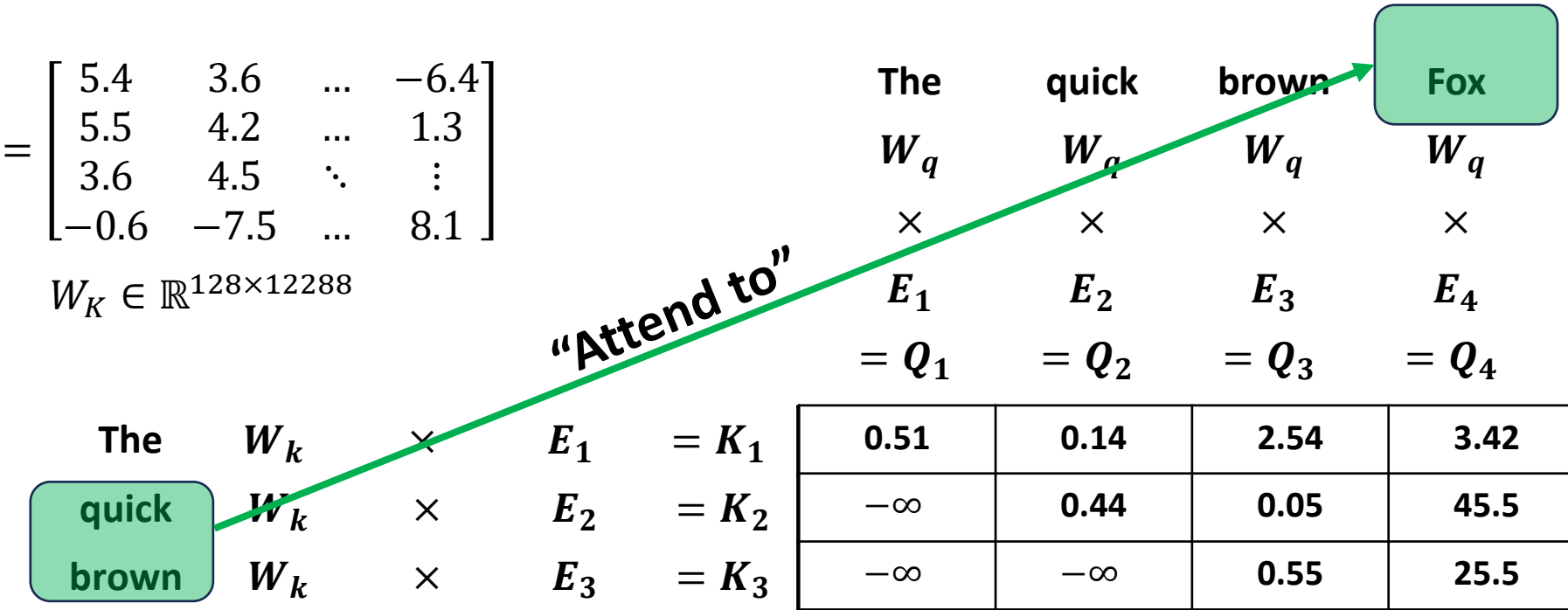


E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

K and Q Computation

$$W_K = \begin{bmatrix} 5.4 & 3.6 & \dots & -6.4 \\ 5.5 & 4.2 & \dots & 1.3 \\ 3.6 & 4.5 & \ddots & \vdots \\ -0.6 & -7.5 & \dots & 8.1 \end{bmatrix}$$

$W_K \in \mathbb{R}^{128 \times 12288}$



$= K^T Q$

$$Attention(Q, K, V) = softmax\left(\frac{K^T Q}{\sqrt{d_k}}\right) V$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Masking

$$W_K = \begin{bmatrix} 5.4 & 3.6 & \dots & -6.4 \\ 5.5 & 4.2 & \dots & 1.3 \\ 3.6 & 4.5 & \ddots & \vdots \\ -0.6 & -7.5 & \dots & 8.1 \end{bmatrix}$$

$W_K \in \mathbb{R}^{128 \times 12288}$

The	quick	brown	Fox
$\boldsymbol{W_q}$	$\boldsymbol{W_q}$	$\boldsymbol{W_q}$	$\boldsymbol{W_q}$
$\times$	$\times$	$\times$	$\times$
$\boldsymbol{E_1}$	$\boldsymbol{E_2}$	$\boldsymbol{E_3}$	$\boldsymbol{E_4}$
$= \boldsymbol{Q_1}$	$= \boldsymbol{Q_2}$	$= \boldsymbol{Q_3}$	$= \boldsymbol{Q_4}$

The	$\boldsymbol{W_k}$	$\times$	$\boldsymbol{E_1}$	$= \boldsymbol{K_1}$
quick	$\boldsymbol{W_k}$	$\times$	$\boldsymbol{E_2}$	$= \boldsymbol{K_2}$
brown	$\boldsymbol{W_k}$	$\times$	$\boldsymbol{E_3}$	$= \boldsymbol{K_3}$
fox	$\boldsymbol{W_k}$	$\times$	$\boldsymbol{E_4}$	$= \boldsymbol{K_1}$

1	0.4934	0.3786	0.0198
0	0.5066	0.3038	0.817
0	0	0.3176	0.1395
0	0	0	0.0237

$$= softmax\left(\frac{K^T Q}{\sqrt{d_k}}\right)$$

$$Attention(Q, K, V) = softmax\left(\frac{K^T Q}{\sqrt{d_k}}\right) V$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

V Computation: Attention Weighting

$$W_V = \begin{bmatrix} 6.1 & 2.5 & \dots & -5.4 \\ 5.5 & 4.2 & \dots & 1.3 \\ 7.6 & -2.3 & \dots & -6.1 \\ 0.17 & 3.2 & \ddots & \vdots \\ -1.6 & -2.5 & \dots & 2.1 \end{bmatrix} \times \begin{bmatrix} 5.4 & 3.6 & \dots & -6.4 \\ 5.5 & 4.2 & \dots & 1.3 \\ 3.6 & 4.5 & \ddots & \vdots \\ -0.6 & -7.5 & \dots & 8.1 \end{bmatrix}$$

$W_{V1} \in \mathbb{R}^{12288\times128}$ $W_{V2} \in \mathbb{R}^{128\times12288}$

The	$W_{V1}\times W_{V2}$	×	$\boldsymbol{E_1}$	$= \boldsymbol{V_1}$	$1\boldsymbol{V_1}$	$0.4934\boldsymbol{V_1}$	$0.3786\boldsymbol{V_1}$	$0.0198\boldsymbol{V_1}$
quick	$W_{V1}\times W_{V2}$	×	$\boldsymbol{E_2}$	$= \boldsymbol{V_2}$	0	$0.5066\boldsymbol{V_2}$	$0.3038\boldsymbol{V_2}$	$0.817\boldsymbol{V_2}$
brown	$W_{V1}\times W_{V2}$	×	$\boldsymbol{E_3}$	$= \boldsymbol{V_3}$	0	0	$0.3176\boldsymbol{V_3}$	$0.1395\boldsymbol{V_3}$
fox	$W_{V1}\times W_{V2}$	×	$\boldsymbol{E_4}$	$= \boldsymbol{V_4}$	0	0	0	$0.0237\boldsymbol{V_4}$

$$= softmax\left(\frac{K^T Q}{\sqrt{d_k}}\right)V$$

$$\boldsymbol{\Delta E_4} = 0.0198\boldsymbol{V_1} + 0.817\boldsymbol{V_2} + 0.1395\boldsymbol{V_3} + 0.0237\boldsymbol{V_4}$$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Multi-Headed Attention

Attention Head 1

$1V_1$	$0.7934V_1$	$0.2786V_1$	$0.0198V_1$
0	$0.2066V_2$	$0.4038V_2$	$0.817V_2$
0	0	$0.3176V_3$	$0.1395V_3$
0	0	0	$0.0237V_4$

Attention Head 2

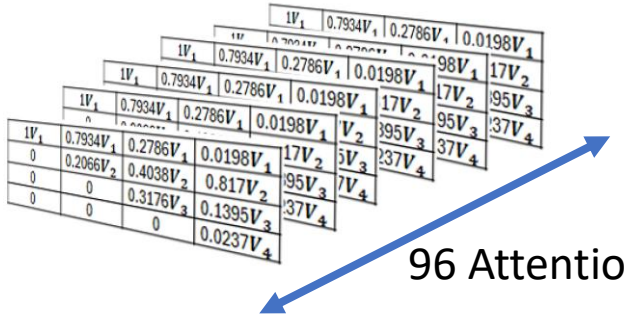
$1V_1$	$0.4934V_1$	$0.3786V_1$	$0.1198V_1$
0	$0.5066V_2$	$0.3038V_2$	$0.817V_2$
0	0	$0.3176V_3$	$0.1395V_3$
0	0	0	$0.0037V_4$

Attention Head 3

$1V_1$	$0.1934V_1$	$0.3786V_1$	$0.0198V_1$
0	$0.8066V_2$	$0.4038V_2$	$0.717V_2$
0	0	$0.2276V_3$	$0.2395V_3$
0	0	0	$0.0237V_4$

...

$$E'_4 = E_4 + \Delta E_4^1 + \Delta E_4^2 + \Delta E_4^3 + \cdots + E_4^{96}$$



96 Attention Heads

$$E' = \begin{bmatrix} 0 \\ 1.3582 \\ 3.3614 \\ \vdots \\ -1.5768 \end{bmatrix} \begin{bmatrix} 0.1093 \\ 0.2442 \\ -0.319 \\ \vdots \\ -0.469 \end{bmatrix} \begin{bmatrix} 0.2299 \\ -0.358 \\ 0.2647 \\ \vdots \\ 0.3884 \end{bmatrix} \begin{bmatrix} -0.245 \\ 0.7354 \\ -6.457 \\ \vdots \\ 2.88 \end{bmatrix}$$

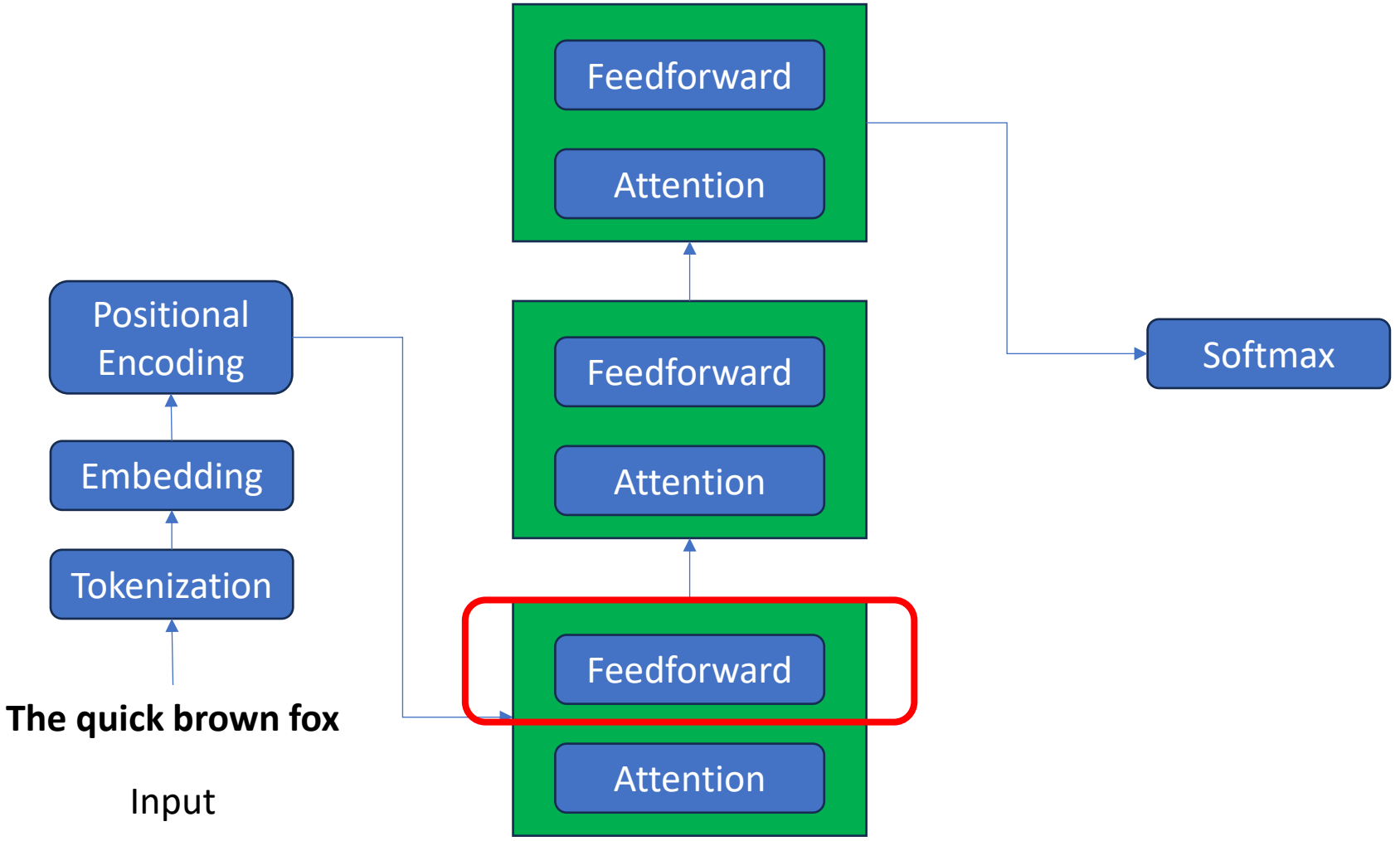
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

GPT-3 175B Parameter Count

Component	Formula	Computation
Embedding	$d_{embed} \times d_{vocab}$	$12{,}288 \times 50{,}257 = 617{,}558{,}016$
Key	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Query	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Value1	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Value2	$d_{embed} \times d_{query} \times n_{heads} \times n_{layers}$	$12{,}288 \times 128 \times 96 \times 96 = 14{,}495{,}514{,}624$
MLP Matrix 1		
MLP Matrix 2		
Unembedding		
	$Total$	

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Multi-Headed Attention

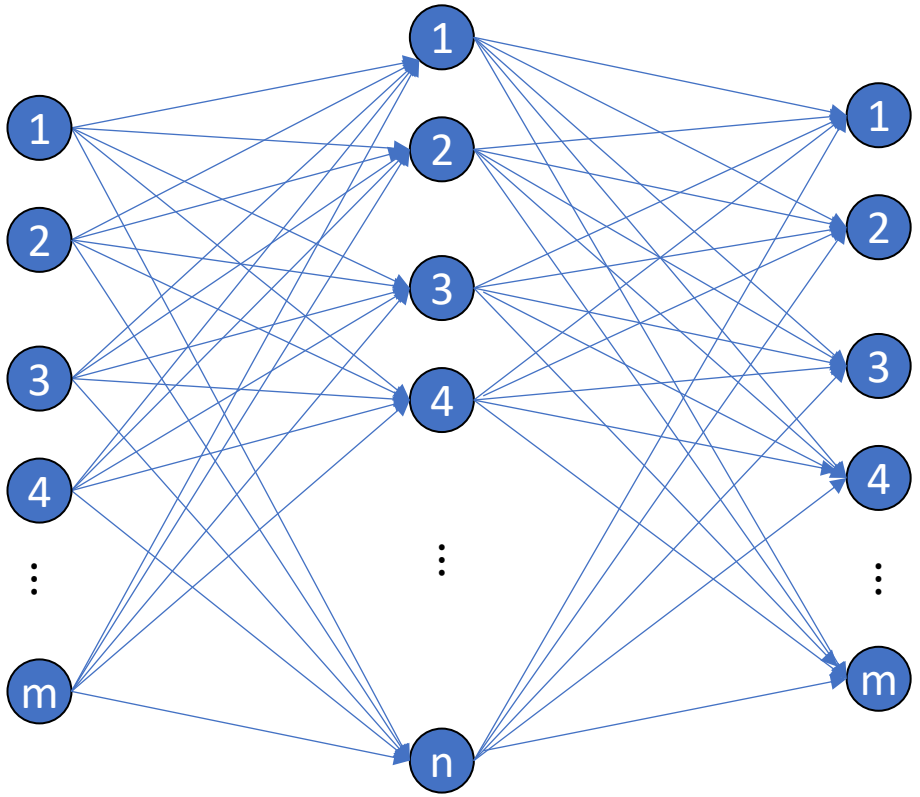


E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Feedforward

12,288 dim

$$\boldsymbol{E'_4} = \begin{bmatrix} -0.545 \\ 0.7454 \\ -6.057 \\ \vdots \\ 1.8688 \end{bmatrix}$$



$$\begin{bmatrix} -1.545 \\ 4.3454 \\ -3.057 \\ \vdots \\ 2.3648 \end{bmatrix} = \boldsymbol{\Delta E'_4}$$

$m = 12{,}288$ nodes and linear activation

$n = 12{,}288 \times 4$ nodes and RELU activation

$m = 12{,}288$ nodes and linear activation

$$\boldsymbol{E''_4} = \boldsymbol{E'_4} + \boldsymbol{\Delta E'_4}$$

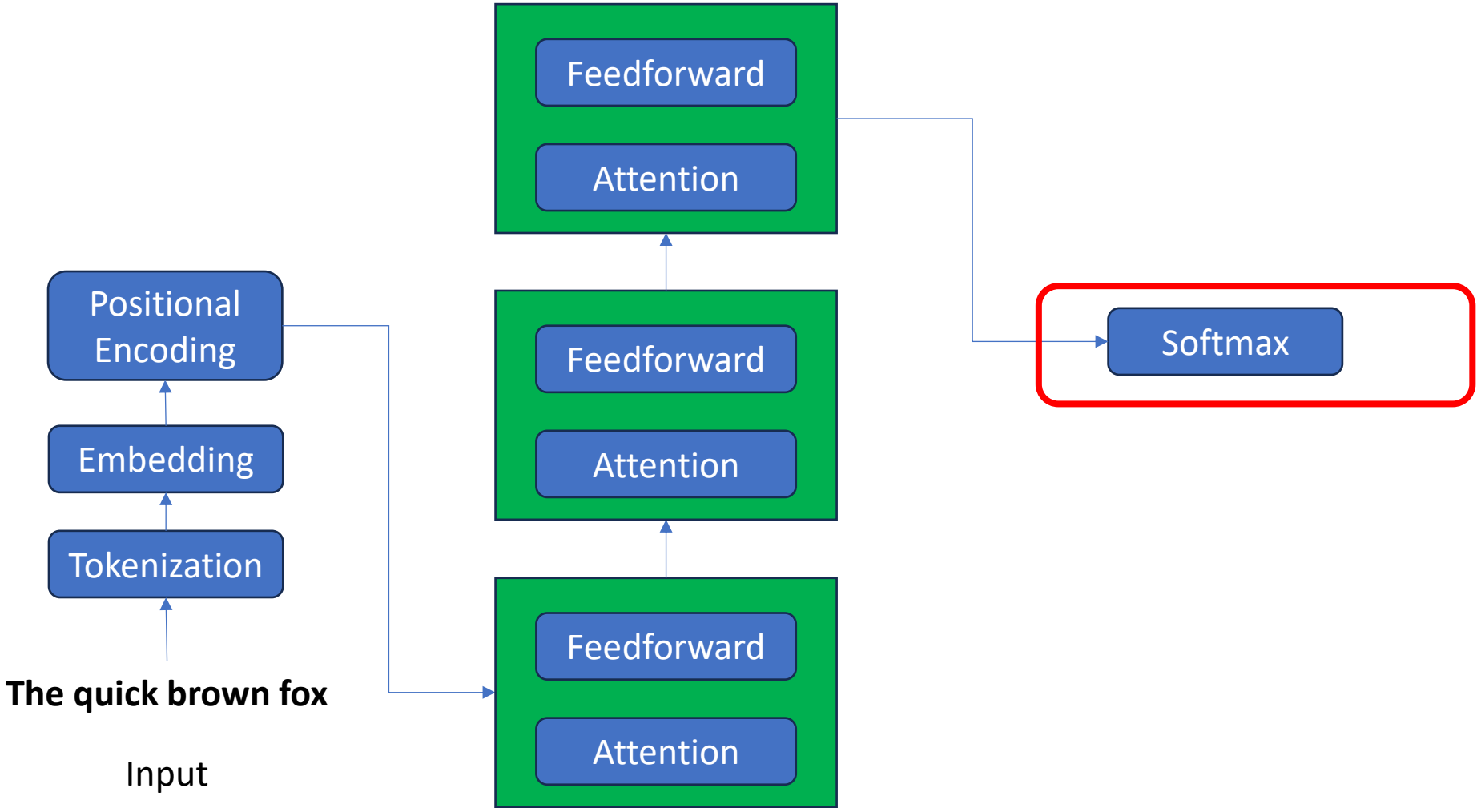
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

GPT-3 175B Parameter Count

Component	Formula	Computation
Embedding	$d_{embed} \times d_{vocab}$	$12{,}288 \times 50{,}257 = 617{,}558{,}016$
Key	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Query	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Value1	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Value2	$d_{embed} \times d_{query} \times n_{heads} \times n_{layers}$	$12{,}288 \times 128 \times 96 \times 96 = 14{,}495{,}514{,}624$
MLP Matrix 1	$d_{embed} \times n_{neurons} \times n_{layers}$	$12{,}288 \times 49{,}152 \times 96 = 57{,}982{,}058{,}496$
MLP Matrix 2	$n_{neurons} \times d_{embed} \times n_{layers}$	$49{,}152 \times 12{,}288 \times 96 = 57{,}982{,}058{,}496$
Unembedding		
	$Total$	

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Multi-Headed Attention



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Unembedding

UnEmbedding Matrix:

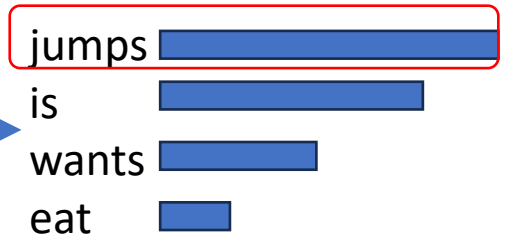
a	0.12	-7.09	-3.17	-4.63	⋮	3.63	7.45
aah	2.9	3.4	-2.45	-0.34	⋮	0	-3.9
aardvark	3.92	0.37	-1.36	1.13	⋮	-1.09	1.09
…	…	…	…	…	⋱	…	…
zygote	4.9	-1.22	0.27	-1.26	⋮	-1.09	-2.17
zymosis	-5.9	-2.39	-1.9	0.2	⋮	-1.36	0.54
zzzz	-2.27	-6.72	2.45	2.372	⋮	-1.36	-1.36

12,288
dim

$$\times \begin{bmatrix} -0.545 \\ 0.7454 \\ -6.057 \\ \vdots \\ 1.8688 \end{bmatrix} = \begin{matrix} E_4'' \end{matrix}$$

a	5.4
aah	5.6
aardvark	3.52
…	…
zygote	2.9
zymosis	-7.9
zzzz	-2.27

Softmax



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

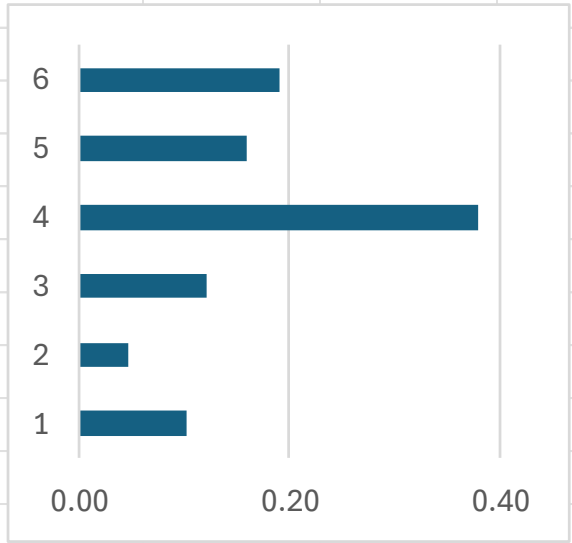
Temperature

- Recall the softmax function: $\phi_i(z_i) = \frac{e^{x_n}}{\sum_j e^{x_j}}$
- With Temperature: $\phi_i(z_i) = \frac{e^{x_n/T}}{\sum_j e^{x_j/T}}$

$$\begin{bmatrix} -0.7 \\ -6.2 \\ 0.5 \\ \mathbf{8.5} \\ 2.4 \\ 3.7 \end{bmatrix} \rightarrow \begin{bmatrix} e^{-0.7} / \sum_{n=1}^{N} e^{x_n} \\ e^{-6.2} / \sum_{n=1}^{N} e^{x_n} \\ e^{0.5} / \sum_{n=1}^{N} e^{x_n} \\ e^{8.5} / \sum_{n=1}^{N} e^{x_n} \\ e^{2.4} / \sum_{n=1}^{N} e^{x_n} \\ e^{3.7} / \sum_{n=1}^{N} e^{x_n} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \\ \mathbf{0.99} \\ 0 \\ 0.01 \end{bmatrix}$$

Temperature	7

Orig	e^x	e^x/sum(e^X)
-0.7	0.90	0.10
-6.2	0.41	0.05
0.5	1.07	0.12
8.5	3.37	0.38
2.4	1.41	0.16
3.7	1.70	0.19



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

GPT-3 175B Parameter Count

Component	Formula	Computation
Embedding	$d_{embed} \times d_{vocab}$	$12{,}288 \times 50{,}257 = 617{,}558{,}016$
Key	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Query	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Value1	$d_{query} \times d_{embed} \times n_{heads} \times n_{layers}$	$128 \times 12{,}288 \times 96 \times 96 = 14{,}495{,}514{,}624$
Value2	$d_{embed} \times d_{query} \times n_{heads} \times n_{layers}$	$12{,}288 \times 128 \times 96 \times 96 = 14{,}495{,}514{,}624$
MLP Matrix 1	$d_{embed} \times n_{neurons} \times n_{layers}$	$12{,}288 \times 49{,}152 \times 96 = 57{,}982{,}058{,}496$
MLP Matrix 2	$n_{neurons} \times d_{embed} \times n_{layers}$	$49{,}152 \times 12{,}288 \times 96 = 57{,}982{,}058{,}496$
Unembedding	$d_{vocab} \times d_{embed}$	$50{,}257 \times 12{,}288 = 617{,}558{,}016$
	$Total$	$175{,}181{,}291{,}520$

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Outline for this Session

- Introduction to ANNs
- Transformers
- Embeddings
- Attention
- **Quantization**
- Project Case

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

IEEE Standard for Floating-Point Arithmetic

- To support a wide range, the default format in most CPUs and GPUs is 32-bit float ($10^{-38}\ to\ 10^{38}$)
- Components of a floating point number
  - Mantissa (m): number of levels
  - Exponent (e): scale to a target range
  - Sign (s): indicates if number is positive or negative

$$(-1)^s \times m \times 2^{(e-127)}$$

  - Single Precision Example
  - $-1.112934 \times 10^{-16}$



$s = 1, \quad e = 74 \qquad\qquad m = 20484$

  - IEEE 754

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Quantization

- **Definition:** **Quantization** reduces the numerical precision used to represent a model's weights—and sometimes activations—to reduce memory requirements and improve inference efficiency.
- A model may originally store weights using:
  - FP32 → 32 bits per parameter
  - FP16 → 16 bits per parameter
  - INT8 → 8 bits per parameter
  - INT4 → 4 bits per parameter
- The objective is:
  - **Represent the model using fewer bits while preserving as much model quality as possible.**

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Quantization

- Model Memory is Driven by Parameter Count and Precision
- A useful approximation is:
  - $Model\ Weight\ Memory \approx Number\ of\ Parameters \times Bits\ per\ Parameter \div 8$
- For a **7-billion-parameter model**:

Precision	Bits / Parameter	Approx. Raw Weight Memory
FP32	32	28 GB
FP16	16	14 GB
INT8	8	7 GB
INT4	4	3.5 GB

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Outline for this Session

- Introduction to ANNs
- Transformers
- Embeddings
- Attention
- Quantization
- **Project Case**

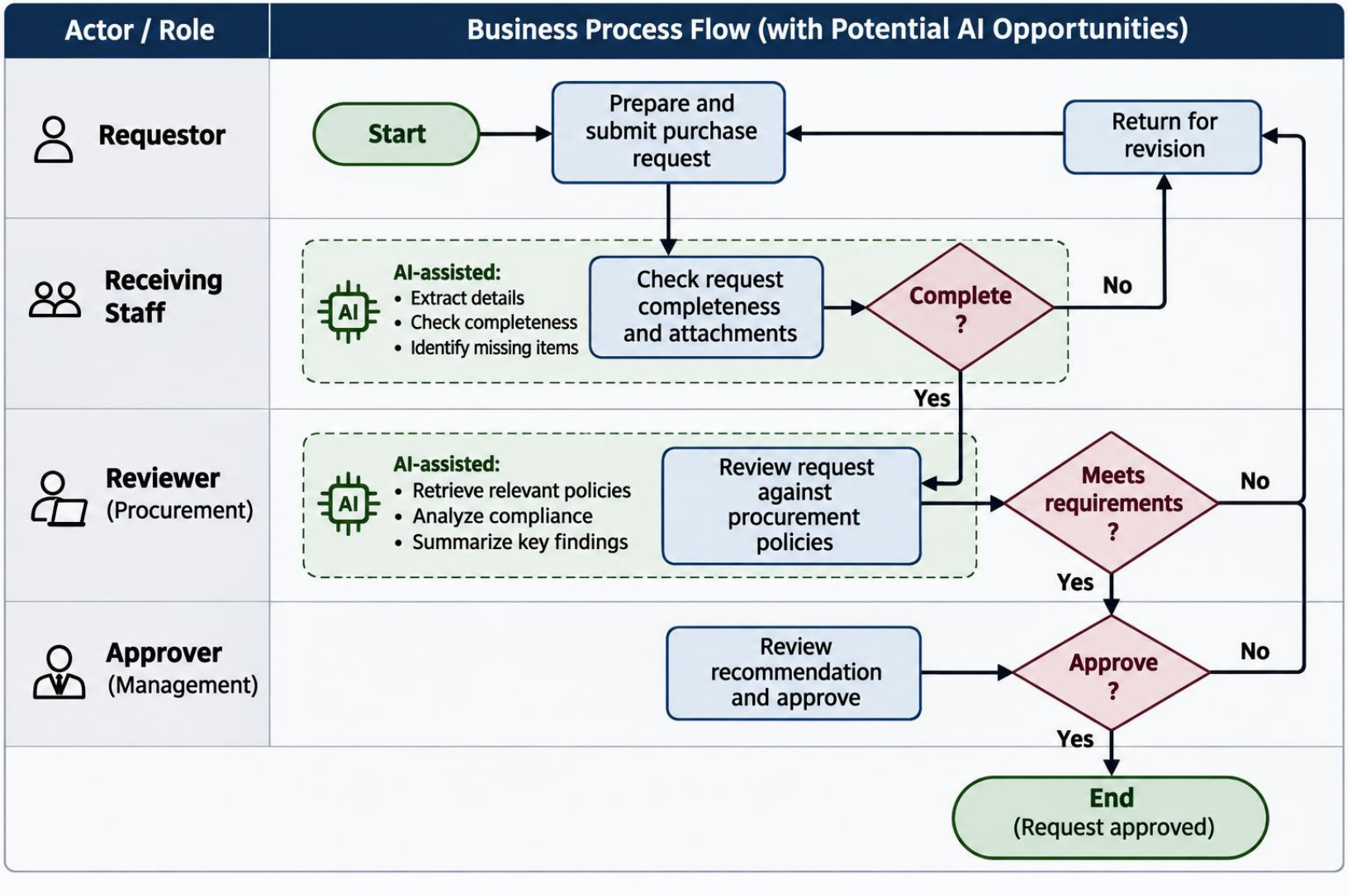
E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Project Case 2: Map the Business Process

- Objective: Develop a swimlane flowchart showing how the proposed business process operates and where AI could potentially provide support.
- Instructions:
  - Review the business problem and initial AI solution proposed in Project Case 1.
  - Identify the actors or departments involved in the business process.
  - Map the major activities performed by each actor, from the initial request to the final outcome.
  - Identify key decision points, approvals, handoffs, and information exchanges.
  - Highlight activities that could potentially be supported or automated by AI.

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Sample Flowchart



E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph

Outline for this Session

- Introduction to ANNs
- Transformers
- Embeddings
- Attention
- Quantization
- Project Case

E.R. L. Jalao, Copyright UP NEC, eljalao@up.edu.ph